PrivAGM: Secure Construction of Differentially Private Directed Attributed Graph Models on Decentralized Social Graphs

Songlei Wang[1], Yifeng Zheng[1], Xiaohua Jia[1,2]
[1]School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen
[2]Department of Computer Science, City University of Hong Kong
songlei.wang@outlook.com,yifeng.zheng@hit.edu.cn,csjia@cityu.edu.hk

ABSTRACT

Decentralized social graphs, where no single entity possesses the information of the entire graph, and each user maintains only a limited view of the graph, contain great value for different applications. However, simply collecting local views for analytics raises privacy concerns due to the sensitive information of social relationships they capture. To address this, a canonical approach involves privately fitting a generative graph model to the decentralized social graph, generating a differentially private synthetic graph that serves as a proxy for analytics. Existing solutions, however, often fail to capture the inherent directionality of edges and attribute-edge correlations when dealing with decentralized directed social graphs, leading to synthetic graphs with poor utility. To bridge this gap, we present PrivAGM, a new solution that harnesses the synergies among differential privacy, secure multiparty computation, and generative graph models, enabling the secure construction of differentially private directed attributed graph models on decentralized social graphs while ensuring the privacy preservation of individuals. We evaluate our solution on three real-world directed social graph datasets. The results demonstrate that PrivAGM outperforms the state-of-the-art methods, generating synthetic graphs with significantly higher utility.

1 INTRODUCTION

With the continuous advancements in graph analytics, a multitude of valuable applications can stem from the in-depth exploration of a social graph, such as the discovery of social roles [13] and mining of interests [4]. However, social graph analytics becomes a challenging issue when the entire graph is not available to a single entity and stored in a *decentralized* manner. In this scenario, each user possesses only a limited local view comprising its attributes and neighbor list in the social graph, and the complete information of the social graph is formed by the collective views of all users. Decentralized social graphs have various practical applications [5, 10, 22, 34, 36]. For example, the phone network exemplifies a decentralized social graph, wherein each user maintains their own profile (i.e., attributes) and contact list (i.e., neighbor list), and the combination of all users' profiles and contact lists constitutes the social graph [34].

In the context of decentralized social graphs, the collection of individual users' local views for analytics can lead to significant privacy concerns, particularly when these local views contain sensitive information about social interactions, political preferences, and religious affiliations [5, 22, 23, 34, 36]. If users' local views are not sufficiently protected, it would discourage them from participating in such analytics. Thus, it is crucial to incorporate security measures into analytics over decentralized social graphs from the outset, facilitating the development and deployment of valuable applications without compromising users' privacy.

Privacy-preserving analytics over decentralized social graphs has received significant attention in recent years [3, 22–24, 34, 36, 43]. Most existing methods [22–24, 36] have adopted local differential privacy (LDP) mechanisms [26] to collect certain noisy statistics from the decentralized social graph for use. However, this line of work suffers from a major deficiency that limits their practical usability. Specifically, as more statistics are collected from the decentralized social graph, the total amount of revealed information monotonically increases, eventually reaching a threshold where collecting any new statistics leads to an excessive privacy risk [12].

Alternatively, a more promising direction is to privately fit a generative graph model to the decentralized social graph, producing a differentially private generative graph model. A synthetic graph is then generated from such model, which emulates the crucial characteristics of the decentralized graph and serves as a reliable proxy for conducting social graph analytics tasks. Based on the post-processing invariance property of differential privacy (DP) [14], the synthetic graph remains differentially private. Hay *et al.* [19] highlight three prominent advantages for utilizing synthetic graphs: (1) Certain analyses require running complex algorithms or simulations over actual social graph data, rather than relying solely on statistics. (2) Synthetic graphs facilitate "exploratory, open-ended, and iterative" analysis, which can be challenging to accommodate directly on an input private graph with limited privacy budget under DP. (3) It eliminates the need to share proprietary analysis details with the social graph owner. Based on these observations, this paper follows the direction of securely constructing differentially private generative graph models on decentralized social graphs.

Existing solutions. In the literature, there have been a number of works considering the setting of *centralized* graphs for the construction of differentially private generative graph models (e.g., [5, 25, 45], to list a few). On the contrary, little work [3, 34, 43, 46] has been done for constructing differentially private generative graph models over *decentralized* social graphs. The works in [3, 34, 46] concentrate solely on modeling graph structure, disregarding vertex attributes and their correlations with the graph structure. Nevertheless, real-world social graphs possess vertex attributes and exhibit such correlations. For example, users often have various attributes (like age, gender, and sexual preference), and they are known to exhibit homophily, wherein users with similar attributes tend to form connections [33]. These correlations can be leveraged in various analyses; for instance, in the field of relational machine learning, they are used to predict missing or future attribute values [25]. The prior work most closely related to ours is AsgLDP [43], which aims

to construct differentially private *attributed* graph models (AGMs) on decentralized social graphs.

However, AsgLDP, as well as other works [3, 34, 46], focuses on idealized decentralized *undirected* social graphs, where each user holds its *complete* neighbor list. In practice, however, decentralized social graphs are more commonly *directed*, where each user holds an *incomplete* neighbor list consisting of only "one-sided relationships". For instance, in phone networks, it is possible that Peter's contact list contains Tom's phone number, while Tom's contact list does not contain Peter's phone number. Another example is the Diaspora application [10], which incorporates the concept of "asymmetric following", enabling users to follow others without requiring mutual consent. Therefore, existing works are not effective in handling such a practical scenario. Additionally, all previous works do not consider the inherent directionality of the edges when constructing the generative graph model, leading to synthetic graphs with poor utility when dealing with directed graphs. Therefore, secure construction of differentially private *directed* AGMs (dAGMs) on decentralized directed social graphs remains to be fully explored.

Our contributions. In this paper, we design, implement, and evaluate PrivAGM, a new solution for secure construction of differentially private dAGMs on decentralized social graphs. PrivAGM is built from synergies among DP, secure multiparty computation (MPC), and generative graph models. We address three challenges when endeavoring to design PrivAGM.

Challenge 1: How to collect neighbor lists while striking a balance between privacy, utility, and efficiency?

To privately collect neighbor lists, a widely-used method is the randomized neighbor list (RNL) mechanism [34], which enables users to flip each bit in their neighbor lists with a certain probability to achieve pure edge LDP. However, the RNL mechanism significantly disrupts the structure of the decentralized social graph, as it requires flipping a considerable number of bits in each neighbor list to achieve the desired level of privacy protection. Another plausible method is to let users encrypt each bit of their neighbor lists using MPC techniques (e.g., additive secret sharing [9]). However, this incurs significant performance overhead as the neighbor lists are normally sparse [7]. To strike a balance between privacy, utility, and efficiency, we instead propose to layer the concept of selective MPC [21] with RNL to encrypt the neighbor lists. Unlike typical secret sharing-based MPC techniques, where all computing parties receive shares for all private values, selective MPC works by randomly selecting a subset of parties to receive shares for each private value. With our sparsity-aware encryption method, PrivAGM achieves pure edge LDP for the neighbor lists *without compromising their utility* or incurring significant performance overhead.

Challenge 2: How to securely extract the differentially private characteristics essential for dAGM from the collected encrypted social graph while ensuring their utility?

Constructing the differentially private dAGM requires the addition of calibrated noises to the extracted characteristics from the original graph. However, for certain characteristics (e.g., triangle subgraph count), the global sensitivity of the calibrated noises is large, which could severely compromise the utility of the extracted characteristics. To overcome this challenge, we develop techniques for oblivious edge clipping, enabling the oblivious restriction of the maximum degree of the encrypted social graph. This helps mitigate the worst-case impact of a single vertex or edge and achieve a small global sensitivity. Moreover, we develop a suite of MPC-based secure components, including secure degree sequence extraction, secure attribute-edge correlation distribution extraction, secure attribute distribution extraction, and secure triangle counts extraction. Their integration allows to securely produce the differentially private characteristics essential for constructing the dAGM.

Challenge 3: How to construct the dAGM to effectively capture the inherent directionality of the edges in the decentralized social graph?

The existing AGMs [5, 25, 33, 43] have mainly focused on *undirected* graphs, resulting in a lack of capability to capture the inherent directionality of edges within the original graph. This directionality is of utmost importance for various analytics tasks on directed graphs, such as the discovery of social roles [13] and prediction of disease outbreaks [25]. In response, we design a directed version of the widely used Chung-Lu (CL) model [6] to effectively incorporate the edge directionality presented in the degrees of vertices. Building upon the directed CL model, we adapt the classical AGM framework [25] to account for the directionality exhibited in the attribute-edge correlations and triangle subgraph counts. This leads to the dAGM, which generates directed attributed graphs that effectively emulate the inherent edge directionality presented in the original graph.

Evaluation results. We conduct experiments on three real-world social graph datasets. The results clearly demonstrate that, in terms of utility of synthetic graphs, PrivAGM not only outperforms the state-of-the-art decentralized setting-based method AsgLDP [43] but also surpasses the state-of-the-art centralized setting-based method CAGM [5]. For instance, in terms of the Hellinger distance between the degree distribution of the synthetic and input social graphs, PrivAGM is 58.3% lower than AsgLDP, and also is 42.9% lower than CAGM, with the privacy budget $\varepsilon = 1$ on the Gplus dataset [29]. We also comprehensively evaluate the system cost. Notably, with $\varepsilon = 1.2$, PrivAGM's sparsity-aware encryption method achieves about 70% savings in ciphertext size over the simple method of encrypting every bit of the sparse neighbor lists.

2 PRELIMINARIES

2.1 Attributed Graphs

This paper focuses on directed and unweighted attributed graphs. An attributed graph is represented as $G = (\mathbf{A}, \mathbf{F})$. Here, $\mathbf{A} \in \{0,1\}^{N \times N}$ is the asymmetric binary adjacency matrix of size $N \times N$, where N is the number of vertices. Each vertex is denoted by $i \in [1, N]$. $\mathbf{A}[i, j] = 1$ indicates the presence of a directed edge from vertex i to vertex j, while $\mathbf{A}[i, j] = 0$ indicates the absence of such a directed edge. We exclude self-connected edges in our consideration, meaning that $\mathbf{A}[i, i] = 0, i \in [1, N]$. The number of edges in G is denoted as M. We use $\mathbf{A}[i, :]$ to denote the i-th row of $\mathbf{A}$, which corresponds to the out-neighbors[1] of vertex i. The number of vertex i's out-neighbors (resp. in-neighbors) is named as its out-degree (resp. in-degree), denoted as d_i^+ (resp. d_i^-). $\mathbf{F} \in \{0,1\}^{N \times L}$ is the attribute matrix, where L is the dimensions of each vertex's attribute vector. $\mathbf{F}[i, :]$ is vertex i's attribute vector. The attribute vectors are assumed to be binary, following previous works [5, 25, 33, 43].

[1]Vertex i's out-neighbors are the vertices reachable by following its outgoing edges, while its in-neighbors are the vertices with directed edges pointing towards it.

We use $\{\sigma_i\}_{i\in[1,\gamma]}$ to denote the set $\{\sigma_1, \cdots, \sigma_\gamma\}$, and omit the subscript $i \in [1, \gamma]$ when it does not impact the presentation.

2.2 Attributed Graph Model (AGM)

We provide an overview of the classical and widely adopted AGM [33] as a foundational approach to demonstrate AGM's underlying principles. Given the original graph, the AGM [33] first learns:

• The degree sequence of all vertices: $\mathcal{D} = \{d_i\}_{i\in[1,N]}$.

• The attribute distribution Θ_F : $\Theta_F(\mathbf{y}) = \frac{n_\mathbf{y}}{N}$, where $n_\mathbf{y}$ is the number of vertices in the graph that have attribute vector $\mathbf{y}$.

• The attribute-edge correlation distribution Θ_X : $\Theta_X(\mathbf{y} - \mathbf{y}') = \frac{n_{\mathbf{y}-\mathbf{y}'}}{M}$, where $\mathbf{y} - \mathbf{y}'$ is named as *attribute-edge correlation* (i.e., an edge connecting two vertices with attribute vector $\mathbf{y}$ and $\mathbf{y}'$, respectively), $n_{\mathbf{y}-\mathbf{y}'}$ is the number of $\mathbf{y} - \mathbf{y}'$ in the original graph.

To generate a graph that emulates the original graph, the AGM first samples an attribute vector for each vertex by Θ_F. It then iteratively samples edges based on the new attributes by $\mathcal{D}$ and Θ_X.

2.3 Differential Privacy for Graphs

In this paper, we consider two variants of DP in the context of attributed graphs: edge LDP [34] and edge DP [25].

Definition 2.1. (Edge LDP [34]) A randomized mechanism $\mathcal{M}$ satisfies ε-edge LDP, if and only if for any two neighbor lists $\mathbf{A}[i,:]$, $\mathbf{A}'[i,:]$ of vertex i that differ in one bit, we have: $\forall \mathbf{b} \in Range(\mathcal{M})$,

$$Pr[\mathcal{M}(\mathbf{A}[i,:]) = \mathbf{b}] \leq e^{\varepsilon} \cdot Pr[\mathcal{M}(\mathbf{A}'[i,:]) = \mathbf{b}],$$

where $Range(\mathcal{M})$ denotes the set of all possible outputs of $\mathcal{M}$, ε is the privacy budget.

Definition 2.2. (Edge DP [25]) A randomized mechanism $\mathcal{M}$ satisfies (ε, δ)-edge DP, if and only if for any two neighboring attributed graphs G and G' that differ in the presence of a single edge or in the attribute associated with a single vertex, we have: $\forall \mathcal{G} \in Range(\mathcal{M})$,

$$Pr[\mathcal{M}(G) = \mathcal{G}] \leq e^{\varepsilon} \cdot Pr[\mathcal{M}(G') = \mathcal{G}] + \delta,$$

where δ is a privacy parameter. If $\delta = 0$, we say that $\mathcal{M}$ provides pure edge DP. It is worth noting that, unlike edge DP in the context of *unattributed* graphs [18], which exclusively addresses edge privacy, edge DP in the context of *attributed* graphs [25] goes a step further to protect the privacy of vertex attributes.

The discrete Laplace distribution $Lap(\cdot)$ is commonly employed to draw discrete noises to provide DP.

Definition 2.3. (Discrete Laplace Distribution [20]) A discrete random variable x follows $Lap(\varepsilon, \delta, \Delta)$ if its probability density function is

$$Pr[x] = \frac{e^{\frac{\varepsilon}{\Delta}} - 1}{e^{\frac{\varepsilon}{\Delta}} + 1} \cdot e^{\frac{-\varepsilon \cdot |x-\mu|}{\Delta}}. \quad (1)$$

where μ is the mean of the distribution and Δ is the sensitivity [14].

2.4 Additive Secret Sharing

In 2-out-of-2 additive secret sharing (ASS) [9], a secret value $x \in \mathbb{Z}_{2^l}$ is split into two secret shares $\langle x\rangle_1$ and $\langle x\rangle_2$, which are distributed to two parties P_1 and P_2, respectively. We denote the ASS of x as $[\![x]\!]$. If only certain values in an entity Ω (e.g., set or table) are secret-shared, we represent the entity as $[\Omega]$. The basic secure operations in the ASS domain are as follows: (1) Linear operations: Linear operations on secret-shared values can be computed locally. Given constants c, c' and secret-shared values $[\![x]\!], [\![y]\!]$, computing $[\![z]\!] = [\![c \cdot x + c' \cdot y]\!]$ involves $P_{i\in\{1,2\}}$ locally computing $\langle z\rangle_i = c \cdot \langle x\rangle_i + c' \cdot \langle y\rangle_i$. (2) Multiplication operations: Multiplication operation on two secret-shared values $[\![x]\!], [\![y]\!]$ requires one round of online communication. To compute $[\![z]\!] = [\![x \cdot y]\!]$, $P_{1,2}$ prepare a secret-shared Beaver triple $([\![w]\!], [\![u]\!], [\![v]\!])$ offline, where $w = u \cdot v$. $P_{i\in1,2}$ first computes $\langle e\rangle_i = \langle x\rangle_i - \langle u\rangle_i$ and $\langle f\rangle_i = \langle y\rangle_i - \langle v\rangle_i$, then communicates with each other to reveal e and f. Finally, $P_{1,2}$ compute $\langle z\rangle_1 = e \cdot f + f \cdot \langle u\rangle_1 + e \cdot \langle v\rangle_1 + \langle w\rangle_1$ and $\langle z\rangle_2 = f \cdot \langle u\rangle_2 + e \cdot \langle v\rangle_2 + \langle w\rangle_2$, respectively.

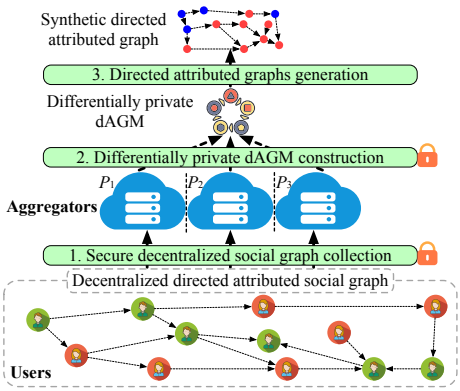


Figure 1: The system architecture of PrivAGM.

3 PROBLEM STATEMENT

3.1 System Architecture

The system architecture of PrivAGM is illustrated in Fig. 1. In the system, there are two types of entities: users and aggregators. The N users, along with the connections among them, collectively form the decentralized social graph $G = (\mathbf{A}, \mathbf{F})$. Each user corresponds to a vertex $i \in [1, N]$ and holds a local view $(\mathbf{A}[i,:], \mathbf{F}[i,:])$, where $\mathbf{A}[i,:]$ represents user i's neighbor list held by it (named as *local neighbor list*) and $\mathbf{F}[i,:]$ represents user i's attribute vectors. In contrast to previous studies [34, 43] that focus on idealized decentralized social graphs where each user's local neighbor list *completely* includes its out/in-neighbors, i.e., $\mathbf{A}$ is symmetric, our attention is directed towards a more realistic *directed* social graph scenario. Here, each user's local neighbor list only includes its out-neighbors[2], i.e., $\mathbf{A}$ is asymmetric. Such decentralized social graphs are more commonly encountered in practice (e.g., the phone networks [34]).

The aggregators aim to collect the decentralized social graph data from the users to perform various graph analytics tasks. However, privacy concerns hinder users' participation in the tasks [34, 45]. Our goal is to facilitate secure construction of a dAGM on the decentralized social graph by the aggregators, while preserving the privacy of individuals. In this paper, we leverage a distributed trust framework where three aggregators (referred to as P_1, P_2, and P_3) from different trust domains cooperatively construct the dAGM. Such framework has recently gained significant attentions in both academia [8, 37, 38, 40] and industry [17, 30, 31].

[2]PrivAGM can be effortlessly expanded to the scenario where each user holds its in-neighbors.

3.2 Threat Model and Security Guarantees

Threat model. We assume that all parties involved, including the aggregators and the users, are honest-but-curious. This means that all parties will adhere to the protocol specification of PrivAGM without deviation, but they may attempt to infer the local views of other users during the execution of PrivAGM. Additionally, we assume that all parties will not collude with each other.

Security guarantees. In the adversary's view, PrivAGM guarantees the following: (1) during the decentralized social graph collection phase, the local neighbor lists are differentially private (edge LDP); (2) during the dAGM construction phase, the learned dAGM is differentially private (edge DP). Note that based on the post-processing invariance of DP [14], the synthetic graphs generated by the differentially private dAGM are still differentially private.

3.3 An Overview of PrivAGM

PrivAGM consists of three phases: (1) secure decentralized social graph collection (Section 4), (2) differentially private dAGM construction (Section 5), and (3) directed attributed graphs generation (Section 6). Their workflow is illustrated in Fig. 1. In phase (1), each user carefully encrypts its local view, followed by distributing the ciphertext to the aggregators. In phase (2), the aggregators securely extract the differentially private parameters necessary for dAGM from the encrypted social graph. Lastly, in phase (3), the aggregators utilize the constructed differentially private dAGM to generate synthetic directed attributed graphs.

4 SECURE DECENTRALIZED SOCIAL GRAPH COLLECTION

We first introduce how each user i encrypts its neighbor list $\mathbf{A}[i,:]$.

Motivation. To encrypt $\mathbf{A}[i,:]$, a simple method is to directly apply lightweight ASS over the entire $\mathbf{A}[i,:]$ of length N (i.e., dense encoding). However, this method is inefficient due to the sparsity of social graphs. According to Facebook's statistics [7], the average user has around 130 friends in a social network, which is much less than the total number of vertices, i.e., N. As a result, $\mathbf{A}[i,:]$ will be mostly filled with zeros, leading to high sparsity. Therefore, applying ASS over the entire $\mathbf{A}[i,:]$ would result in significant uplink communication cost from user i to the aggregators, as well as unnecessary workload during the subsequent dAGM construction on the encrypted social graph. Another simple method is to consider only the nonzero bits in $\mathbf{A}[i,:]$ (i.e., sparse encoding) and apply ASS solely to the indices of these non-zero elements. Specifically, user i first stores $\mathbf{A}[i,:]$ in sparse representation: $E_i = \{(i,j) | \mathbf{A}[i,j] = 1, j \in [1,N]\}$, and then encrypts E_i into $[E_i] = \{(i, [\![j]\!]) | \mathbf{A}[i,j] = 1, j \in [1,N]\}$ ($[E_i] = \{(i, [\![j]\!])\}$ for simplicity). However, the encryption of indices makes it difficult to securely access the neighbors of each user. Since accessing neighbors is a fundamental operation required to extract parameters necessary for dAGM, the method also imposes a heavy workload on the aggregators.

Observation. We observe that while both offering privacy protection, the two aforementioned methods represent two extremes: the first one allows efficient accessing of neighbors but sacrifices sparsity, while the second one preserves sparsity but hinders efficient neighbor accessing. The observation indicates that the challenge here lies in balancing the preservation of privacy and the benefits of sparsity, while facilitating efficient neighbor accessing. We note that employing edge LDP [34] is a strategic approach to address the challenge. To achieve edge LDP, our starting point is the randomized neighbor list (RNL) mechanism. RNL applies randomized response [41], allowing each user to flip each bit of its local neighbor list with a certain probability. However, to achieve the desired level of privacy protection, RNL must flip a considerable number of bits in each local neighbor list, which significantly disrupts the structure of the decentralized social graph. To address this issue, our insight is to layer the concept of selective MPC [21] with RNL to achieve pure edge LDP for the local neighbor lists *without compromising their utility* or incurring significant performance overhead. Unlike typical secret sharing-based MPC techniques [9], where all computing participants receive secret shares for all private values, selective MPC randomly selects a subset of participants to receive secret shares for each private value.

Layering selective MPC with RNL. Given $\mathbf{A}[i,:]$, user i first constructs the set $E_i = \{(i,j,1) | \mathbf{A}[i,j] = 1, j \in [1,N]\}$ to fully capture the nonzero bits in $\mathbf{A}[i,:]$. Next, for every zero bit $\mathbf{A}[i,j] = 0, j \in [1,N]$, user i includes $(i,j,0)$ to E_i *with a probability of* p. The resulting local neighbor list is denoted as $E_i = \{(i,j,b_{ij})\}$, where $b_{ij} = 1$ or 0. User i then applies ASS over each b_{ij} in E_i to obtain the encrypted neighbor list $[E_i] = \{(i,j,[\![b_{ij}]\!])\}$. After that, a straightforward method is to distribute $\{(i,j,\langle b\rangle_1)\}$ and $\{(i,j,\langle b\rangle_2)\}$ to P_1 and P_2, respectively. However, this approach cannot achieve pure edge LDP in the individual view of P_1 or P_2 because it solely *adds* zero bits to E_i and does not *remove* nonzero bits from E_i, thereby capturing only additive noise without accounting for subtractive noise. Our insight to address this issue is to leverage the concept of selective MPC [21]. Specifically, for *each* $(i,j,[\![b_{ij}]\!]) \in [E_i]$, user i sends $(i,j,\langle b\rangle_1)$ and $(i,j,\langle b\rangle_2)$ to two *randomly* selected aggregators, respectively. Thus, each aggregator only has access to a random subset of $[E_i]$. In other words, our method simulates removing some nonzero bits from the view of each aggregator, capturing subtractive noise (similar to how adding zero bits captures additive noise), thereby achieving pure edge LDP in each aggregator's view (as proven in Section 7.2) without compromising the utility.

For each $(i,j,[\![b_{ij}]\!]) \in [E_i]$, user i allows the two selected aggregators to be aware of each other to facilitate subsequent computations in the ASS domain. We use $\{[E_i]^{(1,2)}\}$, $\{[E_i]^{(2,3)}\}$, and $\{[E_i]^{(3,1)}\}$ to represent the encrypted local neighbor lists of all users jointly held by the pairs of aggregators (P_1,P_2), (P_2,P_3), and (P_3,P_1), respectively, and use $\{[E_i]\} = \{[E_i]^{(1,2)} \cup [E_i]^{(2,3)} \cup [E_i]^{(3,1)}\}$ to represent the complete encrypted neighbor lists.

Encrypting the attributes. User i parses its attribute into a value:

$$f_i = \sum_{l \in [1,L]} \mathbf{F}[i,l] \cdot 2^l. \tag{2}$$

User i encrypts f_i into $[\![f_i]\!]$ and distributes $\langle f_i\rangle_1$ to P_1; $\langle f_i\rangle_2$ to P_2.

Finally, the encrypted graph is in the form of $(\{[E_i]\}, \{[\![f_i]\!]\})$.

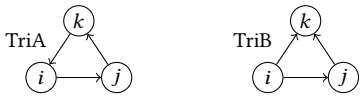
5 DIFFERENTIALLY PRIVATE DAGM CONSTRUCTION

We will now describe the process by which the aggregators securely construct a differentially private dAGM on the encrypted social graph. We first introduce the parameters necessary for our dAGM.

Parameters necessary for dAGM. Our dAGM is built on the widely adopted AGM [33], which involves learning three sets of parameters from the input attributed graph: the degree sequence $\mathcal{D}$, the attribute-edge correlation distribution Θ_X, and the attribute distribution Θ_F (as introduced in Section 2.2). However, the AGM [33] focuses on undirected graphs, whereas PrivAGM specifically addresses directed graphs, where the *inherent directionality* of edges plays a crucial role in mimicking the original graph. Therefore, we extend the AGM [33] by adapting its required parameters, enabling to effectively capture the inherent directionality present in the input directed graph. Since the attribute distribution does not possess directionality, our focus lies solely on adapting the degree sequence and the attribute-edge correlation distribution.

When considering the degree sequence, it is important to note that in directed graphs, each vertex has both out-degree and in-degree. Therefore, to effectively capture the directionality using the degree sequence, we tailor our dAGM to learn the sequence of (out-degree, in-degree) pairs, denoted as $\mathcal{D}^{\pm} = \{(d_i^+, d_i^-)\}_{i\in[1,N]}$. When considering the attribute-edge correlation distribution, to effectively capture the directionality, we tailor our dAGM to learn $\Theta_{\vec{X}} : \Theta_{\vec{X}}(f \to f') = \frac{n_{f\to f'}}{M}, f \to f' \in \mathbb{Z}_{2^L} \times \mathbb{Z}_{2^L}$, where $n_{f\to f'}$ is the number of edges *from* a vertex with attribute value f (mapped from an attribute vector by Eq. 2) *to* a vertex with attribute value f'. The distinction between our $\Theta_{\vec{X}}$ and the AGM [33]'s Θ_X lies in our consideration of attribute-edge correlations $f \to f'$ and $f' \to f$ as distinct, whereas the AGM [33] treats them as equivalent. As a result, our $\Theta_{\vec{X}}$ effectively captures the edge directionality.

The work [25] points out that the count of triangles in the original graph plays a pivotal role in refining the structure of the synthetic graphs. However, it considers all triangles to be identical, thus also failing to capture the directionality of edges. To address this limitation, we propose counting non-isomorphic triangles separately. Specifically, we identify two non-isomorphic triangles "TriA" and "TriB", regardless of the vertex attributes:



Our dAGM counts the number of occurrences of TriA and TriB in the original graph separately, denoted as $n_{\triangle}^A$ and $n_{\triangle}^B$, respectively.

Subsequently, we present our approach for securely extracting the aforementioned four types of parameters (detailed in Sections 5.2, 5.3, 5.4, and 5.5, respectively) from the encrypted social graph. Before detailing the secure parameters extraction, we provide a protocol in Section 5.1 that allows the aggregators to obliviously clip the edges in the encrypted social graph.

5.1 Oblivious Edge Clipping

Motivation. Edge clipping is necessary due to the significant sensitivity Δ involved in achieving edge DP for the attribute-edge correlation distribution $\Theta_{\vec{X}}$ and the triangle counts $n_{\triangle}^A$ and $n_{\triangle}^B$. For example, the addition or removal of a single edge changes $n_{\triangle}^A$ or $n_{\triangle}^B$ by at most $d_{max} - 1$, where d_{max} is the maximum (out or in)-degree in the original graph, which can range up to $N - 1$. Setting such sensitivities and adding noises would completely destroy the utility of the extracted parameters. Hence, to achieve both high utility and edge DP in this scenario, it is crucial to mitigate the worst-case impact of a single vertex or edge. This can be accomplished by applying edge clipping on the original graph to restrict its maximum out/in-degree. A naive approach would be for each user to locally perform edge clipping on their local neighbor lists. However, in the realistic decentralized social graph scenario we are targeting, where each user holds an incomplete neighbor list, it is not feasible for users to effectively restrict their in-degrees.

Algorithm 1 Basic Oblivious Edge Clipping

Input: Encrypted local neighbor lists $\{[E_i]\}$; clipping parameter k.
Output: Encrypted clipped local neighbor lists $\{[\overline{E}_i]\}$.

// $P_{1,2}$ operate on $\{[E_i]^{(1,2)}\}$:
1: Initialization: $[\![d_i^+]\!] = [\![0]\!]; [\![d_i^-]\!] = [\![0]\!]; [\overline{E}_i]^{(1,2)} = \emptyset, i \in [1, N]$.
2: **for all** $(i, j, [\![b_{ij}]\!]) \in [E_i]^{(1,2)}, i \in [1, N]$ **do**
3: $[\![d_i^+]\!] = [\![d_i^+]\!] + [\![b_{ij}]\!]$; $[\![d_j^-]\!] = [\![d_j^-]\!] + [\![b_{ij}]\!]$. // Update the degrees.
4: $[\![\text{isClip}_1]\!] = \text{secComp}([\![d_i^+]\!], k/3)$. // Oblivious comparison.
5: $[\![\text{isClip}_2]\!] = \text{secComp}([\![d_j^-]\!], k/3)$. // Oblivious comparison.
6: $[\![\text{isClip}]\!] = [\![\text{isClip}_1]\!] \cdot [\![\text{isClip}_2]\!]$. // Aggregate the comparison results.
7: $[\![b'_{i,j}]\!] = [\![b_{ij}]\!] \cdot [\![\text{isClip}]\!]$. // Obliviously clip the edge.
8: $[\overline{E}_i]^{(1,2)}.\text{add}((i, j, [\![b'_{i,j}]\!]))$. // Add the updated edge to the new set.
9: **end for**
10: $P_{2,3}$ (resp. $P_{3,1}$) perform the above operations on $\{[E_i]^{(2,3)}\}$ (resp. $\{[E_i]^{(3,1)}\}$) to produce $\{[\overline{E}_i]^{(2,3)}\}$ (resp. $\{[\overline{E}_i]^{(3,1)}\}$).
11: **return** $\{[\overline{E}_i]\} = \{[\overline{E}_i]^{(1,2)} \cup [\overline{E}_i]^{(2,3)} \cup [\overline{E}_i]^{(3,1)}\}$.

Basic oblivious edge clipping. Our (basic) oblivious edge clipping algorithm is built on the widely-used edge clipping algorithm in the plaintext domain [1]. It projects an original graph with arbitrary degrees onto a graph where the maximum degree is k. Given a graph and a clipping parameter k, it iterates through each edge in a random order. If either of the two vertices connected by an edge currently has a degree larger than k, the edge is deleted.

However, the algorithm [1] focuses on undirected graphs, which means it does not differentiate between out-degrees and in-degrees. Hence, we adapt it to accommodate directed graphs. In this context, when provided with a directed graph and a clipping parameter k, our algorithm iterates through each directed edge of the graph in a random order. When considering an edge from vertex i to vertex j, the values of d_i^+, d_i^- are first incremented to 1. After that, if $d_i^+ > k$ or $d_i^- > k$, the edge is deleted. We then detail our secure protocol. As stated in Section 4, each aggregator only holds a subset of each local neighbor list, meaning that the (encrypted) edges held by each aggregator are incomplete. Hence, they cannot simply clip the edges using the clipping parameter k. Our main insight is to empower each pair of aggregators to *independently* perform oblivious edge clipping on their jointly held encrypted local neighbor lists utilizing a clipping parameter of $k/3$. It ensures that, after the edge clipping, the maximum out/in-degree in the (encrypted) subgraph held by each pair of aggregators is restricted to $k/3$. Therefore, the maximum out/in-degree is limited to $k/3 + k/3 + k/3 = k$. Due to the uniform random distribution of edges among $P_{1,2,3}$, this method yields a similar outcome as when clipping edges on complete neighbor lists by parameter k. Algorithm 1 shows the protocol.

Note that $\text{secComp}(\cdot, \cdot)$ (lines 4 and 5) is the oblivious comparison operation. Specifically, $\text{secComp}([\![\alpha]\!], \beta)$ outputs $[\![1]\!]$ if $\alpha \le \beta$ and $[\![0]\!]$ if $\alpha > \beta$. We employ the efficient function secret sharing (FSS) based distributed comparison function (DCF) [2] to

Algorithm 2 Efficient Oblivious Edge Clipping

Input: Encrypted lists $\{[E_i]\}$; clipping parameters k; batch size w.
Output: Encrypted clipped local neighbor lists $\{[\overline{E}_i]\}$.
//$P_{1,2}$ operate on $\{[E_i]^{(1,2)}\}$:
1: Initialization: $[\![d_i^+]\!] = [\![0]\!]; [\![d_i^-]\!] = [\![0]\!]; [\overline{E}_i]^{(1,2)} = \emptyset, i \in [1, N]$.
2: Restrict the out-degree of vertex $i \in [1, N]$ **in parallel**:
3: Initialize a queue $Q_i = \emptyset$.
4: **for all** $(i, j, [\![b_{ij}]\!]) \in [E_i]^{(1,2)}$ **do**
5: $\quad [\![d_i^+]\!] = [\![d_i^+]\!] + [\![b_{ij}]\!]$; $\quad Q_i$.push$((i, j, [\![b_{ij}]\!]))$.
6: $\quad$ **if** the size of the queue $Q_i \geq w$ **then**
7: $\quad\quad [\![\text{isClip}]\!] = \text{secComp}([\![d_i^+]\!], k/3)$.
8: $\quad\quad$ **while** $Q_i \neq \emptyset$ **do**
9: $\quad\quad\quad (i, j, [\![b_{ij}]\!]) = Q_i$.pop().
10: $\quad\quad\quad [\![b'_{i,j}]\!] = [\![b_{ij}]\!] \cdot [\![\text{isClip}]\!]$. // Obliviously clip the edge.
11: $\quad\quad\quad [\overline{E}_i]^{(1,2)}$.add$((i, j, [\![b'_{i,j}]\!]))$. // Add updated edge to new set.
12: $\quad\quad$ **end while**
13: $\quad$ **end if**
14: **end for**
15: Restrict the in-degrees on $[\overline{E}_i]^{(1,2)}$ **in parallel** as above.
16: $P_{2,3}$ (resp. $P_{3,1}$) perform the above operations on $\{[E_i]^{(2,3)}\}$ (resp. $\{[E_i]^{(3,1)}\}$) to produce $\{[\overline{E}_i]^{(2,3)}\}$ (resp. $\{[\overline{E}_i]^{(3,1)}\}$).
17: **return** $\{[\overline{E}_i]\} = \{[\overline{E}_i]^{(1,2)} \cup [\overline{E}_i]^{(2,3)} \cup [\overline{E}_i]^{(3,1)}\}$.

instantiate it. At line 6, isClip $= 1$ means that $\text{isClip}_1 = 1$ and $\text{isClip}_2 = 1$, indicating that $d_i^+ \leq k/3$ and $d_j^- \leq k/3$, and thus the edge $(i, j, [\![b_{ij}]\!])$ does not need to be clipped. isClip $= 0$ indicates that either $\text{isClip}_1 = 0$ or $\text{isClip}_2 = 0$, meaning that either $d_i^+ > k/3$ or $d_j^- > k/3$, and thus the edge $(i, j, [\![b_{ij}]\!])$ needs to be deleted. Finally, $P_{1,2}$ execute the oblivious edge clipping operation (line 7). The design intuition is that if isClip $= 0$, b'_{ij} is assigned 0, i.e., the edge is clipped, while if isClip $= 1$, b'_{ij} is assigned b_{ij}, i.e., the edge remains unchanged. Likewise, $P_{2,3}$ (resp. $P_{3,1}$) perform above operations on $\{[E_i]^{(2,3)}\}$ (resp. $\{[E_i]^{(3,1)}\}$).

Efficient oblivious edge clipping. While Algorithm 1 can achieve oblivious edge clipping, it comes with a notable communication round complexity. Specifically, as Algorithm 1 iterates through each edge *sequentially*, its communication rounds grow linearly with the number of edges. To address this issue, we propose an efficient protocol based on Algorithm 1, as given in Algorithm 2. The enhancement comes from the independence of the out-degree (resp. in-degree) of each vertex from the out-degree (resp. in-degree) of other vertices. Hence, Algorithm 2 enables the aggregators to simultaneously restrict each vertex's out/in-degree *in parallel* (lines 2 and 15). Another performance improvement from our proposed *batch edge clipping* technique. We observe that the true edges in the encrypted local neighbor lists are sparse. This means that directly clipping the edges in a batch after a single check of the current degree (incremented by the edges in the batch), will seldom lead to erroneously clipping true edges. Hence, to reduce the number of required oblivious comparison operations, we modify the procedure by enabling the aggregators to check whether the current encrypted out-degree or in-degree is larger than $k/3$ after iterating w edges (i.e., a batch), instead of performing the check after iterating each edge. The edges in each batch are then obliviously updated without distinction based on the encrypted comparison result.

Algorithm 3 Secure Degree Sequence Extraction

Input: Encrypted clipped local neighbor lists $\{[\overline{E}_i]\}$; maximum out/in-degree k; privacy budget $\varepsilon_{\mathcal{D}}$; privacy parameter $\delta_{\mathcal{D}}$.
Output: Differentially private degree sequence $\widetilde{\mathcal{D}}^{\pm}$.
// P_1, P_2, and P_3 locally count the encrypted degrees:
Initialization: $[\![d_i^+]\!] = [\![0]\!]; [\![d_i^-]\!] = [\![0]\!], i \in [1, N]$.
$[\![d_i^+]\!] = [\![d_i^+]\!] + [\![b_{ij}]\!]$, $[\![d_j^-]\!] = [\![d_j^-]\!] + [\![b_{ij}]\!]$, $(i, j, [\![b_{ij}]\!]) \in \{[\overline{E}_i]\}$.
P_3 secret-shares its shares of degrees with $P_{1,2}$.
$P_{1,2}$ add the received shares from P_3 to their own shares to obtain $[\![d_i^+]\!], [\![d_i^-]\!], i \in [1, N]$ in 2-out-of-2 ASS.
$P_{1,2}$ arrange the encrypted degrees into table $[\![\mathbf{T}]\!]$, where $[\![\mathbf{T}[i, :]]\!] = [\![d_i^+]\!] \| [\![d_i^-]\!] \| [\![\rho]\!], i \in [1, N]$ and $\rho = 1$.
// P_3 constructs dummy degree pair table $\mathbf{T}'$:
$\mathbf{T}' = [\,]$. // Empty table.
for all $(d^+, d^-), d^+ \in [0, k], d^- \in [0, k]$ **do**
$\quad n = \max(0, Lap(\varepsilon_{\mathcal{D}}, \delta_{\mathcal{D}}, 4))$. // Draw a noise from $Lap(\varepsilon_{\mathcal{D}}, \delta_{\mathcal{D}}, 4)$.
$\quad$ Append n rows of $d^+ \| d^- \| \rho$ to $\mathbf{T}'$, where $\rho = 0$.
end for
P_3 secret-shares $\mathbf{T}'$ with $P_{1,2}$ to produce $[\![\mathbf{T}']\!]$.
// $P_{1,2}$ perform the remaining operations:
$[\![\hat{\mathbf{T}}]\!] = \frac{[\![\mathbf{T}]\!]}{[\![\mathbf{T}']\!]}$. // Vertical concatenation.
$[\![\hat{\mathbf{T}}']\!] = \text{secShuffle}([\![\hat{\mathbf{T}}]\!])$. // Oblivious row-wise shuffle.
Safely reveal columns 1 and 2 of $[\![\hat{\mathbf{T}}']\!]$ to obtain $[\hat{\mathbf{T}}']$.
Initialize encrypted degree histogram $[\![\mathbf{H}]\!] = [\![0]\!]^{(k+1)\times(k+1)}$.
for all $d^+ \| d^- \| [\![\rho]\!]$ in $[\hat{\mathbf{T}}']$ **do**
$\quad [\![\mathbf{H}[d^+, d^-]]\!] = [\![\mathbf{H}[d^+, d^-]]\!] + [\![\rho]\!]$. // Index edge to histogram $[\![\mathbf{H}]\!]$.
end for
$[\![\widetilde{\mathbf{H}}[d^+, d^-]]\!] = [\![\mathbf{H}[d^+, d^-]]\!] + [\![Lap(\varepsilon_{\mathcal{D}}, 0, 4)]\!], d^+, d^- \in [0, k]$.
Safely reveal the differentially private histogram $\widetilde{\mathbf{H}}$.
$\widetilde{\mathbf{H}}'[d^+, d^-] = \max(0, \widetilde{\mathbf{H}}[d^+, d^-]), d^+, d^- \in [0, k]$. // Truncation.
$\widetilde{\mathbf{H}}''[d^+, d^-] = \widetilde{\mathbf{H}}'[d^+, d^-] \cdot \frac{N}{\text{sum}(\widetilde{\mathbf{H}}')}, d^+, d^- \in [0, k]$ // Normalization.
Initialize degree sequence $\widetilde{\mathcal{D}}^{\pm} = \emptyset$.
$\forall d^+, d^- \in [0, k]$, add $\widetilde{\mathbf{H}}''[d^+, d^-]$ tuples (d^+, d^-) to $\widetilde{\mathcal{D}}^{\pm}$.
return Differentially private degree sequence $\widetilde{\mathcal{D}}^{\pm}$.

5.2 Secure Degree Sequence Extraction

Design rationale. We give the method (Algorithm 3) by which $P_{1,2,3}$ securely extract the noisy version of the degree sequence $\mathcal{D}^{\pm}$ from the encrypted clipped local neighbor lists $\{[\overline{E}_i]\}$, while ensuring that the extracted noisy sequence (denoted as $\widetilde{\mathcal{D}}^{\pm}$) is differentially private (edge DP). A naive method is to simply add secret-shared discrete Laplace noise to each encrypted out/in-degree, followed by opening the noisy degrees to obtain $\widetilde{\mathcal{D}}^{\pm}$. However, this method introduces a significant amount of noise, particularly in low-degree vertices, which are abundant in real social graphs [25]. To tackle this issue, our solution is based on the insight that the mapping between a specific vertex and its degree is unimportant for how dAGM uses the degree sequence. What we actually need is the sequence of degrees, and the order is unimportant. Hence, $P_{1,2,3}$ extract the differentially private *histogram* of (out-degree, in-degree) pairs and then derive a more precise $\widetilde{\mathcal{D}}^{\pm}$ from the histogram.

Secure degree histogram construction. $P_{1,2,3}$ first construct the encrypted histogram $[\![\mathbf{H}]\!]$. We observe that the primary challenge in constructing $[\![\mathbf{H}]\!]$ lies in securely indexing each encrypted degree pair $([\![d_i^+]\!], [\![d_i^-]\!])$ within $[\![\mathbf{H}]\!]$. To achieve this, our insight is to let $P_{1,2}$ obliviously shuffle the encrypted degree pairs based on

a random permutation unknown to them to break the mappings between the degree pairs and the vertices, and then securely reveal the shuffled degree pairs to index them in $[\![\mathbf{H}]\!]$.

However, simply revealing the encrypted degree pairs after the oblivious shuffle will directly expose $\mathbf{H}$. To address this, we let P_3 secret-share a certain number of dummy entities $d^+ \| d^- \| \rho$ (with flag $\rho = 0$) for each possible degree pair $(d^+, d^-), d^+ \in [0, k], d^- \in [0, k]$ with $P_{1,2}$ (lines 6-11). $P_{1,2}$ then perform the oblivious row-wise shuffle protocol [16] (denoted as secShuffle$(\cdot)$) on the real degree pairs (with flag $[\![\rho]\!] = [\![1]\!]$) together with the dummy degree pairs (with flag $[\![\rho]\!] = [\![0]\!]$), followed by revealing all degree pairs (without revealing the flags; lines 12-14). The oblivious shuffle ensures that $P_{1,2}$ cannot distinguish between the dummy and real degree pairs. To ensure the revealed frequency of degree pairs in the views of $P_{1,2}$ is differentially private, the number of dummy entities for each possible degree pair is drawn from the discrete Laplace distribution $Lap(\varepsilon_{\mathcal{D}}, \delta_{\mathcal{D}}, 4)$ and truncated to 0 (line 8). Here, the sensitivity Δ is set to 4, as the addition or removal of an edge changes the out/in-degrees of exactly two vertices, resulting in changes to the frequency of at most four degree pairs. We will prove in Section 7.2 that the revealed degree pairs are differentially private in the individual view of P_1, P_2, even if the noises drawn by P_3 may be truncated to 0. Note that we cannot use the revealed frequency of degree pairs as $\mathbf{H}$ due to P_3 knowing the number of dummy entities for each degree pair, i.e., the frequency is not differentially private in P_3's view. Instead, $P_{1,2}$ add the encrypted flag of each plaintext degree pair to the corresponding element of $[\![\mathbf{H}]\!]$ (lines 15-18).

Secure degree sequence construction. $P_{1,2}$ then construct $\widetilde{\mathcal{D}}^{\pm}$ based on $[\![\mathbf{H}]\!]$. $P_{1,2}$ first add a secret-shared noise drawn from $Lap(\varepsilon_{\mathcal{D}}, 0, 4)$ to each element of $[\![\mathbf{H}]\!]$ (line 19). The secret-shared noises can be generated offline by the protocol described in Section 4 of [15]. $P_{1,2}$ safely reveal the differentially private histogram $[\![\widetilde{\mathbf{H}}]\!]$ to obtain $\widetilde{\mathbf{H}}$. $\widetilde{\mathcal{D}}^{\pm}$ can be constructed based on $\widetilde{\mathbf{H}}$ after truncation and normalization (lines 21-24). It will be proven in Section 7.2 that $\widetilde{\mathcal{D}}^{\pm}$ is differentially private in the individual view of P_1, P_2, and P_3.

5.3 Secure Attribute-Edge Correlation Distribution Extraction

We now present how $P_{1,2,3}$ securely extract the noisy version of attribute-edge correlation distribution $\Theta_{\vec{X}}$, while ensuring that the extracted noisy distribution (denoted as $\widetilde{\Theta}_{\vec{X}}$) is differentially private. Algorithm 4 gives the protocol. Its design intuition shares similarities with Algorithm 3. Hence, we omit its introduction. Note that the edges held by each aggregator are incomplete. In addressing this problem, out insight is to let a specific pair of aggregators (assumed to be $P_{1,2}$) to compute $\widetilde{\Theta}_{\vec{X}}$ on their jointly held (incomplete) local neighbor lists. The uniform random distribution of edges among the three aggregators allows this approach to produce a distribution similar to that based on complete local neighbor lists. Note that the sensitivity Δ is set to $2 \cdot k/3$ (lines 7 and 16), as modifying the attribute value of a single vertex can affect, at most, $k/3$ edges (which corresponds to the maximum degree in $[\overline{E}_i]^{(1,2)}$), resulting in changes to the frequency of $2 \cdot k/3$ attribute-edge correlations.

5.4 Secure Attribute Distribution Extraction

We present how $P_{1,2,3}$ securely extract the noisy version of attribute distribution Θ_F from $\{[\![f_i]\!]\}$, while ensuring that the extracted noisy distribution (denoted as $\widetilde{\Theta}_F$) is differentially private in the individual view of $P_{1,2,3}$. The algorithm is given in Appendix A of our full version [39]. The privacy budget and parameter are denoted as ε_F and δ_F, respectively.

Algorithm 4 Secure Attribute-Edge Correlation Distribution Extraction

Input: Encrypted clipped local neighbor lists $\{[\overline{E}_i]\}$; encrypted attributes $\{[\![f_i]\!]\}$; maximum degree k; privacy budget, parameter ε_X, δ_X.
Output: Differentially private attribute-edge distribution $\widetilde{\Theta}_{\vec{X}}$.

// $P_{1,2}$ operate on $\{[\overline{E}_i]^{(1,2)}\}$ and $\{[\![f_i]\!]\}$:
1: $[\![\mathbf{T}]\!] = [\,]$. // Empty table.
2: **for all** $(i, j, [\![b_{ij}]\!]) \in [\overline{E}_i]^{(1,2)}, i \in [1, N]$ **do**
3: Append row $[\![f_i]\!] \| [\![f_j]\!] \| [\![\rho]\!]$ to $[\![\mathbf{T}]\!]$, where $[\![\rho]\!] = [\![b_{ij}]\!]$.
4: **end for**
// P_3 constructs dummy attribute-edge correlation table $\mathbf{T}'$:
5: $\mathbf{T}' = [\,]$. // Empty table.
6: **for all** $f \to f' \in \mathbb{Z}_{2^L} \times \mathbb{Z}_{2^L}$ **do**
7: $n = \max(0, Lap(\varepsilon_X, \delta_X, 2 \cdot k/3))$. // Draw a noise.
8: Append n rows of $f \| f' \| \rho$ to $\mathbf{T}'$, where $\rho = 0$.
9: **end for**
10: P_3 secret-shares $\mathbf{T}'$ with $P_{1,2}$ to produce $[\![\mathbf{T}']\!]$.
// $P_{1,2}$ perform the remaining operations:
11: $[\![\hat{\mathbf{T}}]\!] = \frac{[\![\mathbf{T}]\!]}{[\![\mathbf{T}']\!]}$. // Vertical concatenation.
12: $[\![\hat{\mathbf{T}}']\!] = \text{secShuffle}([\![\hat{\mathbf{T}}]\!])$. // Oblivious row-wise shuffle
13: Safely reveal columns 1 and 2 of $[\![\hat{\mathbf{T}}']\!]$ to obtain $[\hat{\mathbf{T}}']$.
14: Initialize encrypted histogram $[\![\mathbf{H}]\!] = [\![0]\!]^{2^L \times 2^L}$.
15: **for all** row $f \| f' \| [\![\rho]\!]$ in $[\hat{\mathbf{T}}']$ **do**
16: $[\![\mathbf{H}[f, f']]\!] = [\![\mathbf{H}[f, f']]\!] + [\![\rho]\!]$. // Index correlation to $[\![\mathbf{H}]\!]$.
17: **end for**
18: $[\![\widetilde{\mathbf{H}}[f, f']]\!] += [\![Lap(\varepsilon_X, 0, 2 \cdot k/3)]\!], f \to f' \in \mathbb{Z}_{2^L} \times \mathbb{Z}_{2^L}$.
19: Safely reveal the differentially private histogram $\widetilde{\mathbf{H}}$.
20: $\widetilde{\mathbf{H}}'[f, f'] = \max(0, \widetilde{\mathbf{H}}[f, f']), f \to f' \in \mathbb{Z}_{2^L} \times \mathbb{Z}_{2^L}$. // Truncation.
21: $\widetilde{\mathbf{H}}''[f, f'] = \frac{\widetilde{\mathbf{H}}'[f, f']}{\text{sum}(\widetilde{\mathbf{H}}')}, f \to f' \in \mathbb{Z}_{2^L} \times \mathbb{Z}_{2^L}$. // Normalization.
22: Initialize attribute-edge correlation distribution $\widetilde{\Theta}_{\vec{X}} = \{0\}^{2^L \times 2^L}$.
23: $\widetilde{\Theta}_{\vec{X}}(f \to f') = \widetilde{\mathbf{H}}''[f, f'], f \to f' \in \mathbb{Z}_{2^L} \times \mathbb{Z}_{2^L}$.
24: **return** Differentially private attribute-edge distribution $\widetilde{\Theta}_{\vec{X}}$.

5.5 Secure Triangle Counts Extraction

We then present the method (Algorithm 5) by which $P_{1,2,3}$ extract the noisy version of triangle counts $n_{\triangle}^A$ and $n_{\triangle}^B$, while ensuring that the extracted noisy counts (denoted as $\widetilde{n}_{\triangle}^A$ and $\widetilde{n}_{\triangle}^B$) are differentially private. The underlying idea is to enable $P_{1,2,3}$ to iterate over every possible set of three connected edges in pairs, incrementing the encrypted product of flags for these three edges to the encrypted triangle count. However, since secure multiplication in the secret sharing domain necessitates communication between the two involved parties, the aforementioned method can only count triangles whose three edges are held by the *same* pair of aggregators. To address this issue, our solution is based on the observation: the probability of the three edges of each triangle being distributed to the same pair of aggregators is $3 \cdot \frac{1}{3} \cdot \frac{1}{3} \cdot \frac{1}{3} = \frac{1}{9}$. Therefore, PrivAGM enables each pair of aggregators to count triangles over the edges they jointly hold. The encrypted triangle counts are then multiplied by 9 to obtain approximate triangle counts. Note that the sensitivity Δ is set to $k - 1$ (lines 20-21) since adding or removing a single edge can change the triangle count $n_{\triangle}^A$ or $n_{\triangle}^B$ by at most $k - 1$.

Algorithm 5 Secure Triangle Counts Extraction

Input: Encrypted clipped local neighbor lists $\{[\overline{E}_i]\}$; privacy budget ε_T; maximum out/in-degree k.
Output: Differentially private triangle counts $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$.

1: Initialize $[\![n^A_\triangle]\!] = [\![0]\!]$; $[\![n^B_\triangle]\!] = [\![0]\!]$ in 3-out-of-3 ASS.
// $P_{1,2}$ operate on $\{[\overline{E}_i]^{(1,2)}\}$:
// Securely count TriA:
2: **for all** $i : i \in [1, N]$ **in parallel do**
3: **for all** $j : j < i$ and $(i, j, [\![b_{ij}]\!]) \in [\overline{E}_i]^{(1,2)}$ **do**
4: **for all** $k : k < i$ and $(j, k, [\![b_{jk}]\!]) \in [\overline{E}_j]^{(1,2)}$ **do**
5: **if** $(k, i, [\![b_{ki}]\!]) \in [\overline{E}_k]^{(1,2)}$ **then**
6: $[\![n^A_\triangle]\!] = [\![n^A_\triangle]\!] + [\![b_{ij}]\!] \cdot [\![b_{jk}]\!] \cdot [\![b_{ki}]\!]$. // Update the count.
7: **end if**
8: **end for**
9: **end for**
10: **end for**
// Securely count TriB:
11: **for all** $i : i \in [1, N]$ **in parallel do**
12: **for all** $j, k : j \neq k$ and $(i, j, [\![b_{ij}]\!]), (i, k, [\![b_{ik}]\!]) \in [\overline{E}_i]^{(1,2)}$ **do**
13: **if** $(j, k, [\![b_{jk}]\!]) \in [\overline{E}_j]^{(1,2)}$ **then**
14: $[\![n^B_\triangle]\!] = [\![n^B_\triangle]\!] + [\![b_{ij}]\!] \cdot [\![b_{ik}]\!] \cdot [\![b_{jk}]\!]$. // Update the count.
15: **end if**
16: **end for**
17: **end for**
18: $P_{2,3}$ (resp. $P_{3,1}$) count triangles on $\{[\overline{E}_i]^{(2,3)}\}$ (resp. $\{[\overline{E}_i]^{(3,1)}\}$) as above and aggregate the encrypted triangle counts to $[\![n^A_\triangle]\!]$ and $[\![n^B_\triangle]\!]$.
19: $[\![n^A_\triangle]\!] = 9 \cdot [\![n^A_\triangle]\!]$; $[\![n^B_\triangle]\!] = 9 \cdot [\![n^B_\triangle]\!]$.
20: $[\![\widetilde{n}^A_\triangle]\!] = [\![n^A_\triangle]\!] + [\![Lap(\varepsilon_T, 0, k-1)]\!]$. //Add secret-shared noise.
21: $[\![\widetilde{n}^B_\triangle]\!] = [\![n^B_\triangle]\!] + [\![Lap(\varepsilon_T, 0, k-1)]\!]$. //Add secret-shared noise.
22: Safely reveal the differentially private triangle counts $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$.
23: **return** Differentially private triangle counts $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$.

Remark. We acknowledge that performing Algorithm 5 sequentially would entail a significant number of communication rounds. However, as the iterations in the two outermost loops (i.e., lines 2 and 11) are independent of each other, we can let the aggregators execute them in parallel. Moreover, we observe that the primary cost arises from the secure 3-input multiplication gates in lines 6 and 14. To speed up Algorithm 5, we use the technique outlined in Section B.3.1 of [32] to reduce the online communication cost of each secure 3-input multiplication gate from 2 rounds and 8 elements to 1 round and 6 elements. Additionally, we employ a random sampling strategy to further reduce the cost. Specifically, instead of iterating over each vertex, we only iterate over η randomly sampled vertices, and then multiply the encrypted triangle counts by $\frac{N}{\eta}$.

6 DIRECTED ATTRIBUTED GRAPHS GENERATION

We now introduce how the extracted dAGM parameters can be utilized to generate directed attributed graphs. Our algorithm is built upon the classical AGM framework [25]. However, as previously mentioned, it focuses on undirected graphs. Hence, our primary contribution is to adapt the AGM framework [25] to effectively cater for the requirements of generating directed graphs.

Overview. Fig. 2 gives an overview of our algorithm. We note that the Chung-Lu (CL) model is a widely used basic generative graph models, known for its effective emulation of the degree distribution of the original graph. However, the original CL model is designed exclusively for undirected graphs. Therefore, we propose a directed version of the CL model, which is used to sample an initial *directed* edge set $E^{(0)} = \{(i, j)\}$ based on $\widetilde{\mathcal{D}}^{\pm}$. Next, Algorithm 6 is employed to rewire the edges to ensure that the counts of TriA and TriB formed by $E^{(0)}$ are no less than $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$, respectively. The rewired edges are denoted as $E'^{(0)}$. At the same time, the vertex attribute set $\{f_i\}_{i\in[1,N]}$ (where f_i for vertex $i \in [1, N]$) is sampled from $\widetilde{\Theta}_F$. The accept/reject sampling technique is then applied to generate new edge sets. Specifically, the edge acceptance probabilities $\Theta^{(0)}_{Ac}$ are computed based on the current generated graph $(E'^{(0)}, \{f_i\})$ and $\widetilde{\Theta}_{\vec{X}}$. Then a new edge set $E^{(1)}$ is sampled by the directed CL model and a variant of Algorithm 6 considering $\Theta^{(0)}_{Ac}$. This process iterates until Θ_{Ac} converges.

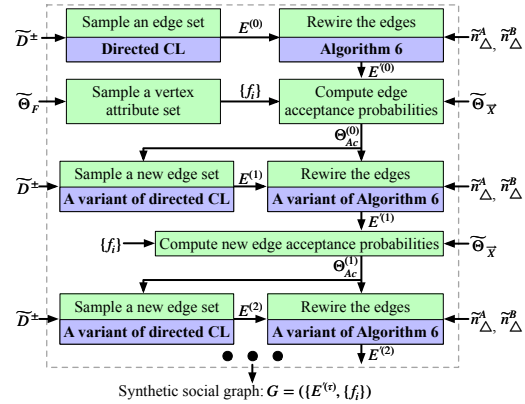


Figure 2: Overview of directed attributed graphs generation.

6.1 Directed CL Model

In the original CL model [6], a synthetic graph is generated by two steps. Firstly, each vertex is assigned a desired degree based on the degree sequence $\{d_i\}_{i\in[1,N]}$ extracted from the original graph. Secondly, $\sum_{i\in[1,N]} d_i$ edges are sampled using a probability that is proportional to the desired degrees of the endpoint vertices. Specifically, an edge between vertices i, j is sampled with a probability of $\frac{d_i \cdot d_j}{\sum_{i\neq j, i,j\in[1,N]} d_i \cdot d_j}$. However, we observe that the model cannot capture the inherent directionality of edges present in the original *directed* graph. To overcome this limitation, we propose the *directed CL model* (denoted as directedCL$(\cdot)$).

In our directed CL model , we first assign each vertex $i \in [1, N]$ a desired out/in-degree pair (d^+_i, d^-_i) based on $\widetilde{\mathcal{D}}^{\pm}$. Then, for each directed edge from vertex i to vertex j, we sample it with a probability proportional to the out-degree of vertex i and the in-degree of vertex j, i.e., $\frac{d^+_i \cdot d^-_j}{\sum_{i\neq j, i,j\in[1,N]} d^+_i \cdot d^-_j}$. If the edge (i, j) is selected, (i, j) will be included in the edge set $E^{(0)}$. To maintain the expected degree distribution, we sample $\max(\sum_{i\in[1,N]} d^+_i, \sum_{i\in[1,N]} d^-_i)$ edges. Clearly, our model can capture the edge directionality in the original graph by sampling each edge based on the out-degree of the edge's starting vertex and the in-degree of the edge's ending vertex.

6.2 Refining Graph Structure with Edge Rewiring

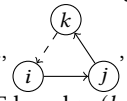
The (unattributed) graphs generated by the directed CL model only capture the degree information of the original graph, which

Algorithm 6 Refining Graph Structure with Edge Rewiring

Input: The edge set E; degree sequence $\widetilde{\mathcal{D}}^{\pm}$; triangle counts $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$.
Output: The new edge set E' after rewiring.

1: $n^A_\triangle, n^B_\triangle = \mathsf{countTriangle}(E)$. // Count triangles on E.
2: **while** $n^A_\triangle < \widetilde{n}^A_\triangle$ and $n^B_\triangle < \widetilde{n}^B_\triangle$ **do**
3: // Construct TriA:
4: $i = \mathsf{Sample}([1, N], \widetilde{\mathcal{D}}^{\pm})$. // Sample a vertex based on $\widetilde{\mathcal{D}}^{\pm}$.
5: $j = \mathsf{SampleNbr}(E, i)$. // Randomly sample a vertex from vertex i's out-neighbors.
6: $k = \mathsf{SampleNbr}(E, j)$.
7: **if** $(k, i) \notin E$ **then**
8: $(q, r) = \mathsf{oldestEdge}(E)$. // Get the oldest edge from E.
9: **if** replacing (q, r) by (k, i) boosts triangle counts **then**
10: $E.\mathsf{replace}((q, r), (k, i))$. // Replace (q, r) by (k, i).
11: Make edge (k, i) the youngest edge in E.
12: $n^A_\triangle, n^B_\triangle = \mathsf{countTriangle}(E)$.
13: **else**
14: Make edge (q, r) the youngest edge in E.
15: **end if**
16: **end if**
// Construct TriB:
17: $i = \mathsf{Sample}([1, N], \widetilde{\mathcal{D}}^{\pm})$.
18: $j, k = \mathsf{Sample2Nbr}(E, i)$. // Randomly sample two vertices from vertex i's out-neighbors.
19: If $(j, k) \notin E$, replacing the oldest edge by (j, k) as above.
20: **end while**
21: **return** The edge set $E' = E$ after rewiring.

is insufficient to simulate the structure of the original graph. To address this limitation, the AGM [25] proposes an iterative edge rewiring method to ensure that the number of triangles in the generated graph is no less than that of the original graph, thereby enhancing the structural representation of the generated graph. However, when considering directed graphs, the AGM overlooks the inherent directionality of edges in the original graph and treats all triangles as identical. To overcome this limitation in the context of directed graphs, we propose Algorithm 6 (denoted as $\mathsf{Rewire}(\cdot)$).

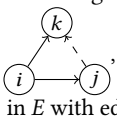
Algorithm 6 operates on the edge set E generated by the directed CL model. It iteratively rewires the edges until the counts of TriA and TriB formed by the edges surpass $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$, respectively. Specifically, to create a new edge for constructing TriA (lines 4-16), the algorithm proceeds as follows: 1) sample a vertex i from $[1, N]$ where each vertex $i \in [1, N]$ is sampled with a probability of $\frac{d^+_i}{\sum_{i\in[1,N]} d^+_i}$; 2) randomly sample a vertex j from vertex i's out-neighbors; 3) randomly sample a vertex k from vertex j's out-neighbors. If $(k, i) \notin E$, indicating that vertices i, j, and k form an incomplete TriA, i.e.,  , the algorithm attempts to replace the oldest edge in E by edge (k, i) to construct TriA (lines 8-15). Specifically, if replacing the oldest edge (q, r) in E with edge (k, i) would increase both the counts of TriA and TriB, edge (q, r) is replaced by edge (k, i) and (k, i) is set as the youngest edge in E. Otherwise, edge (q, r) is set as the youngest edge in E.

Similarly, Algorithm 6 creates a new edge to construct TriB as follows (lines 17-19). First, a vertex i is sampled from $[1, N]$ where each vertex $i \in [1, N]$ is chosen with a probability of $\frac{d^+_i}{\sum_{i\in[1,N]} d^+_i}$.

Algorithm 7 Directed Attributed Graphs Generation

Input: The dAGM parameters $\widetilde{\mathcal{D}}^{\pm}$, $\widetilde{\Theta}_{\vec{X}}$, $\widetilde{\Theta}_F$, $\widetilde{n}^A_\triangle$, and $\widetilde{n}^B_\triangle$.
Output: The synthetic directed social graph $G = (E, \{f_i\})$.

1: $\{f_i\}_{i\in[1,N]} = \mathsf{Sample}(\widetilde{\Theta}_F)$. // Sample a vertex attribute set.
2: $E^{(0)} = \mathsf{directedCL}(\widetilde{\mathcal{D}}^{\pm})$. // Sample the initial edge set.
3: $E'^{(0)} = \mathsf{Rewire}(E^{(0)}, \widetilde{\mathcal{D}}^{\pm}, \widetilde{n}^A_\triangle, \widetilde{n}^B_\triangle)$. // Rewire the edges by Algorithm 6.
4: Initialize the round $\tau = 0$.
5: **while** edge acceptance probability Θ_{Ac} does not converage **do**
6: Compute $\Theta^{(\tau)}_{\vec{X}}$ of the current graph $(E'^{(\tau)}, \{f_i\})$.
7: $R(f \to f') = \frac{\widetilde{\Theta}_{\vec{X}}(f\to f')}{\Theta^{(\tau)}_{\vec{X}}(f\to f')}, f \to f' \in \mathbb{Z}_{2^L} \times \mathbb{Z}_{2^L}$.
8: **if** $\tau > 0$ **then**
9: $R(f \to f') = R(f \to f') \cdot \Theta^{(\tau-1)}_{Ac}(f \to f'), f \to f' \in \mathbb{Z}_{2^L} \times \mathbb{Z}_{2^L}$.
10: **end if**
11: $\Theta^{(\tau)}_{Ac}(f \to f') = \frac{R(f\to f')}{\mathsf{SUP}(R)}, f \to f' \in \mathbb{Z}_{2^L} \times \mathbb{Z}_{2^L}$.
// Sample a new edge set $E'^{(\tau+1)}$ based on $\Theta^{(\tau)}_{Ac}$:
12: $E^{(\tau+1)} = \mathsf{directedCL}(\widetilde{\mathcal{D}}^{\pm}, \{f_i\}, \Theta^{(\tau)}_{Ac})$.
13: $E'^{(\tau+1)} = \mathsf{Rewire}(E^{(\tau+1)}, \widetilde{\mathcal{D}}^{\pm}, \widetilde{n}^A_\triangle, \widetilde{n}^B_\triangle, \{f_i\}, \Theta^{(\tau)}_{Ac})$.
14: $\tau = \tau + 1$.
15: **end while**
16: **return** $E = E'^{(\tau)}$; The synthetic social graph $G = (E, \{f_i\})$.

Then, two vertices j and k are randomly sampled from vertex i's out-neighbors. If $(j, k) \notin E$, indicating that vertices i, j, and k form an incomplete TriB, i.e.,  , the algorithm checks if replacing the oldest edge (q, r) in E with edge (j, k) would increase both the counts of TriA and TriB. If it does, the algorithm replaces edge (q, r) with edge (j, k) and makes (j, k) the youngest edge in E. Otherwise, it makes edge (q, r) the youngest edge in E.

6.3 Putting It Together

Algorithm 7 gives the end-to-end workflow for synthesizing directed graphs. Lines 6-11 calculate the new acceptance probabilities $\Theta^{(\tau)}_{Ac}$ based on the attribute-edge correlation $\Theta^{(\tau)}_{\vec{X}}$ in the current graph $(E'^{(\tau)}, \{f_i\})$ and $\widetilde{\Theta}_{\vec{X}}$ derived from the original graph. $\mathsf{SUP}(R)$ is the least upper bound of set R. The new edge set $E'^{(\tau+1)}$ is sampled using the modified structural model (lines 12-13). Specifically, when the directed CL model or Algorithm 6 add an edge (i, j) to the new edge set, they first sample a value uniformly from the range of 0 to 1 ($\mathsf{Sample}(0, 1)$). (i, j) is included in the new edge set (i.e., "accepted") only when $\mathsf{Sample}(0, 1) \le \Theta^{(\tau+1)}_{Ac}(f_i \to f_j)$, where f_i and f_j are the attributes sampled for vertices i and j at line 1. If $\mathsf{Sample}(0, 1) > \Theta^{(\tau+1)}_{Ac}(f_i \to f_j)$, the edge (i, j) is not included in the new edge set (i.e., "rejected").

7 PRIVACY AND SECURITY ANALYSIS

In Section 7.1, we parametrize the leakage function $\mathcal{L}$, which indicates the additional knowledge that the adversary may acquire during the construction of dAGM. In Section 7.2, we conduct a formal analysis to prove that the output of $\mathcal{L}$ is differentially private. In Section 7.3, we prove that the adversary's view can be simulated solely based on the output of $\mathcal{L}$. Since the users can only observe their local views and the final output of the differentially private

computation (received from the aggregators), we will specifically focus on the situation where the adversary controls one of $P_{\{1,2,3\}}$.

7.1 Leakage Functions

We define the leakage functions as $\mathcal{L} = (\mathcal{L}_E, \mathcal{L}_{\mathcal{D}}, \mathcal{L}_X, \mathcal{L}_F, \mathcal{L}_\triangle)$:

• $\mathcal{L}_E = \{[E_i]\}$, where $[E_i] = \{(i, j, \langle b_{ij} \rangle)\}$ denotes the local neighbor list held by the adversary for user i.

• $\mathcal{L}_{\mathcal{D}} = ([\hat{\mathbf{T}}'_{\mathcal{D}}], \widetilde{\mathbf{H}}_{\mathcal{D}})$, where $[\hat{\mathbf{T}}'_{\mathcal{D}}]$ and $\widetilde{\mathbf{H}}_{\mathcal{D}}$ are the table and histogram revealed in lines 14 and 20 of Algorithm 3, respectively. While the adversary acquires additional information during the execution of the remaining operations in Algorithm 3, since this information is derived from $\widetilde{\mathbf{H}}_{\mathcal{D}}$, it is not included in $\mathcal{L}_{\mathcal{D}}$.

• $\mathcal{L}_X = ([\hat{\mathbf{T}}'_X], \widetilde{\mathbf{H}}_X)$, where $[\hat{\mathbf{T}}'_X]$ and $\widetilde{\mathbf{H}}_X$ are the table and histogram revealed in lines 13 and 19 of Algorithm 4, respectively.

• $\mathcal{L}_F = ([\hat{\mathbf{T}}'_F], \widetilde{\mathbf{H}}_F)$, where $[\hat{\mathbf{T}}'_F]$ and $\widetilde{\mathbf{H}}_F$ are the table and histogram revealed in secure attribute distribution extraction.

• $\mathcal{L}_\triangle = (\widetilde{n}^A_\triangle, \widetilde{n}^B_\triangle)$, where $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$ are the triangle counts revealed in line 22 of Algorithm 5.

7.2 Privacy Analysis

Theorem 7.1. *The local neighbor list $[E_i] \in \mathcal{L}_E$ of user $i \in [1, N]$ held by the adversary adheres to pure ε_E-edge LDP with $\varepsilon_E = \max\{\ln(1/p), \ln(3 - 2p)\}$.*

We provide the proof of Theorem 7.1 in Appendix B of our full version [39].

Theorem 7.2. *Given $\mathcal{L}_{\mathcal{D}}$, in the view of the adversary, $[\hat{\boldsymbol{T}}'_{\mathcal{D}}]$ adheres to $(\varepsilon_{\mathcal{D}}, \delta_{\mathcal{D}})$-edge DP and $\widetilde{\boldsymbol{H}}_{\mathcal{D}}$ adheres to pure $\varepsilon_{\mathcal{D}}$-edge DP.*

We provide the proof of Theorem 7.2 in Appendix C of our full version [39].

Theorem 7.3. *Given $\mathcal{L}_X$, in the view of the adversary, $[\hat{\boldsymbol{T}}'_X]$ adheres to $(\varepsilon_X, \delta_X)$-edge DP and $\widetilde{\boldsymbol{H}}_X$ adheres to pure ε_X-edge DP.*

Proof. Since the proof follows the same idea as the proof of Theorem 7.2, we omit the proof. □

Theorem 7.4. *Given $\mathcal{L}_F$, in the view of the adversary, $[\hat{\boldsymbol{T}}'_F]$ adheres to $(\varepsilon_F, \delta_F)$-edge DP and $\widetilde{\boldsymbol{H}}_F$ adheres to pure ε_F-edge DP.*

Proof. Since the proof also follows the same idea as the proof of Theorem 7.2, we omit the proof. □

Theorem 7.5. *Given $\mathcal{L}_\triangle = (\widetilde{n}^A_\triangle, \widetilde{n}^B_\triangle)$, in the view of the adversary, both $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$ adhere to pure $\varepsilon_\triangle$-edge DP.*

We provide the proof of Theorem 7.5 in Appendix D of our full version [39].

7.3 Security Analysis

We use the simulation paradigm [28] to analyze the security.

Definition 7.6. Let $\prod$ denote the protocol for constructing dAGM on a decentralized social graph G. $\mathsf{View}^{\mathsf{Real}}_{\prod(G)}$ denotes the view of $\mathcal{A}$ in the real-world experiment. In the ideal world, a probabilistic polynomial-time simulator $\mathcal{S}$ generates a simulated view $\mathsf{View}^{\mathsf{Ideal}}_{\mathcal{S},\mathcal{L}}$ for $\mathcal{A}$ using only $\mathcal{L}$. If $\mathsf{View}^{\mathsf{Ideal}}_{\mathcal{S},\mathcal{L}}$ is indistinguishable from $\mathsf{View}^{\mathsf{Real}}_{\prod(G)}$, we conclude that $\prod$ is secure.

Theorem 7.7. *PrivAGM is secure based on Definition 7.6.*

We provide the proof of Theorem 7.7 in Appendix E of our full version [39].

8 PERFORMANCE EVALUATION

8.1 Setup

We develop a prototype implementation of PrivAGM using a combination of Python and C++. All experiments are conducted on a workstation equipped with 24 Intel Xeon Gold 6240R CPU cores, a NVIDIA RTX A6000 GPU, 128 GB of RAM, and 2 TB of external SSD storage, running Ubuntu 20.04.3 LTS. Additionally, we use a MacBook Air with 8 GB of RAM to simulate users, encrypt the local views, and distribute them to the workstation. Our experiments consider a WAN setting with a maximum throughput of 40MB/s and an RTT latency of 40 ms. All public and private values are encoded in $\mathbb{Z}_{2^{32}}$. Given the overall privacy budget ε, we allocate $\varepsilon_{\mathcal{D}} = 0.6\varepsilon, \varepsilon_X = 0.1\varepsilon, \varepsilon_F = 0.1\varepsilon, \varepsilon_\triangle = 0.1\varepsilon$. In addition, we set the privacy parameters $\delta_{\mathcal{D}} = \delta_X = \delta_F = 10^{-5}$. The clipping parameter k follows the previous work [25] and is set to $\sqrt[3]{N}$. The clipping batch size w is set to $\sqrt{N}$, and the sampling number η in the two outermost loops of Algorithm 5 is set to $\sqrt{N}$.

Datasets. We use three real-world directed social graph datasets: MOOC (7047 vertices; 411749 edges) [27], Twitter (81306 vertices; 1768149 edges) [29], and Gplus (107614 vertices; 13673453 edges) [29]. We consider ten attributes, which is significantly more than the previous works [25, 33], who only consider two attributes.

Baselines. We consider two state-of-the-art solutions, CAGM [5] and AsgLDP [43], as the baselines for utility comparison in attributed graph synthesis. To ensure fairness, we use the recommended parameters from the original papers of the baselines.

Utility metrics. We evaluate the utility of the synthetic graphs using four widely-used metrics [5, 25, 34, 43]. Their computation method can be found in Appendix F of our full version [39].

• *Degree distribution:* We use the Kolmogorov-Smirnov statistic to evaluate how well a synthetic graph captures the degree distribution of the original graph (denoted as $\mathsf{KS}_{\mathsf{degree}}$). As $\mathsf{KS}_{\mathsf{degree}}$ is less sensitive to differences in the tails of the degree distributions, we also report the Hellinger distance (denoted as $\mathsf{HD}_{\mathsf{degree}}$) between the degree distributions of the synthetic and original graphs.

• *Attribute-edge correlation distribution:* We report the mean relative error (MRE) (denoted as $\mathsf{MRE}_{\mathsf{corr}}$) and the Hellinger distance $\mathsf{HD}_{\mathsf{corr}}$ between the attribute-edge correlation distribution of the synthetic and original graphs.

• *Local clustering coefficient:* We report the MRE between the average of the local clustering coefficient of the synthetic and original graphs (denoted as $\mathsf{MRE}_{\mathsf{cluster}}$).

• *Triangle counts:* We report the MRE in the number of triangles, TriA and TriB, between the synthetic and original graphs (denoted as $\mathsf{MRE}_{\mathsf{TriA}}$ and $\mathsf{MRE}_{\mathsf{TriB}}$ respectively).

8.2 Utility Comparison

Degree distribution. Fig. 3 presents a comparison between PrivAGM and the baselines in terms of $\mathsf{KS}_{\mathsf{degree}}$ and $\mathsf{HD}_{\mathsf{degree}}$. It is evident that AsgLDP shows the largest $\mathsf{KS}_{\mathsf{degree}}$ and $\mathsf{HD}_{\mathsf{degree}}$ since it is based on LDP, which introduces larger noise than central DP under the same privacy budget. In contrast, our PrivAGM achieves the smallest $\mathsf{KS}_{\mathsf{degree}}$ and $\mathsf{HD}_{\mathsf{degree}}$. This is attributed to PrivAGM preserving the inherent edge directionality in the input graphs.

Attribute-edge correlation distribution. Fig. 4 shows a comparison between PrivAGM and the baselines in terms of $\mathsf{MRE}_{\mathsf{corr}}$

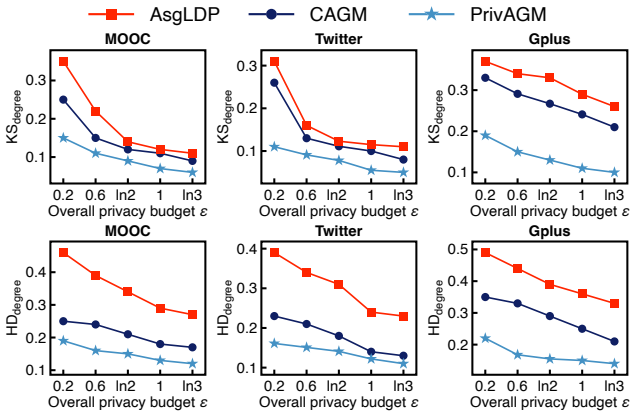


Figure 3: Comparison of PrivAGM and the baselines in terms of $\mathsf{KS}_{\mathsf{degree}}$ and $\mathsf{HD}_{\mathsf{degree}}$, for varying the privacy budget ε.

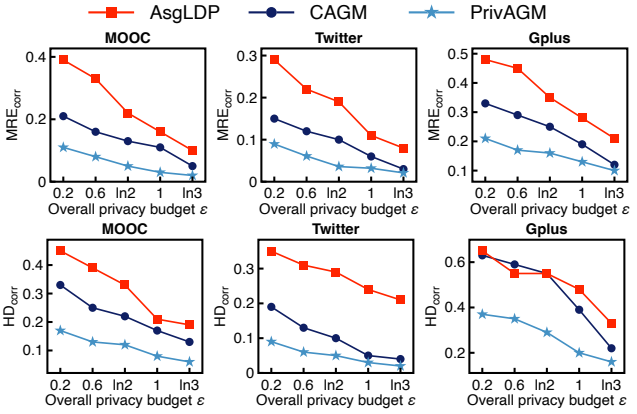


Figure 4: Comparison of PrivAGM and the baselines in terms of $\mathsf{MRE}_{\mathsf{corr}}$ and $\mathsf{HD}_{\mathsf{corr}}$, for varying the privacy budget ε.

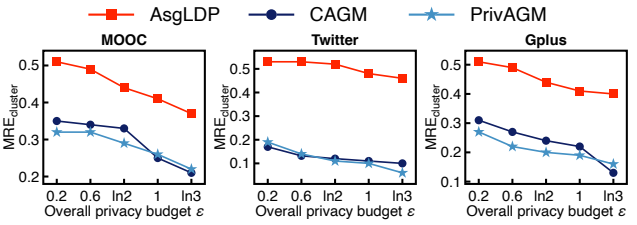


Figure 5: Comparison of PrivAGM and the baselines in terms of $\mathsf{MRE}_{\mathsf{cluster}}$, for varying the privacy budget ε.

and $\mathsf{HD}_{\mathsf{corr}}$. PrivAGM consistently achieves the smallest MREcorr and HDcorr across all privacy budgets and datasets. Moreover, as ε decreases, the difference in MREcorr, HDcorr between PrivAGM and the baselines becomes more significant.

Local clustering coefficient. Fig. 5 presents a comparison between PrivAGM and the baselines in terms of $\mathsf{MRE}_{\mathsf{cluster}}$. It can be observed that the $\mathsf{MRE}_{\mathsf{cluster}}$ of PrivAGM and CAGM is approximately similar. In certain instances (e.g., $\varepsilon = \ln 3$ on the Gplus dataset), the $\mathsf{MRE}_{\mathsf{cluster}}$ of CAGM is even lower than that of PrivAGM. This disparity is because $\mathsf{MRE}_{\mathsf{cluster}}$ fails to capture the difference in edge directionality between the synthetic and the

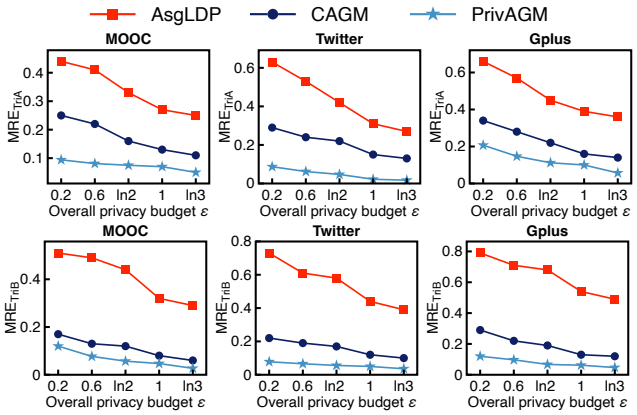


Figure 6: Comparison of PrivAGM and the baselines in terms of $\mathsf{MRE}_{\mathsf{TriA}}$ and $\mathsf{MRE}_{\mathsf{TriB}}$, for varying the privacy budget ε.

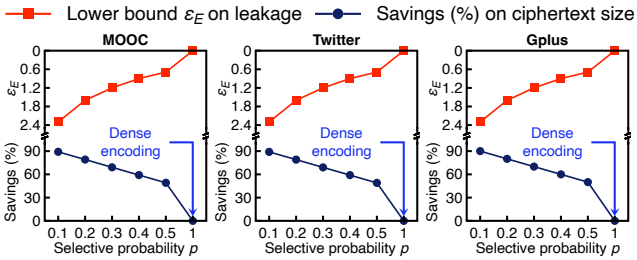


Figure 7: Evaluation of ε_E for leakage in neighbor list encryption and the savings on the ciphertext size, for varying p.

original graphs. Hence, the advantage of PrivAGM in capturing the directionality of the original graphs cannot be fully reflected by $\mathsf{MRE}_{\mathsf{cluster}}$.

Triangle counts. Fig. 6 presents a comparison between PrivAGM and the baselines in terms of $\mathsf{MRE}_{\mathsf{TriA}}$ and $\mathsf{MRE}_{\mathsf{TriB}}$. The results demonstrate that PrivAGM consistently achieves a significantly small $\mathsf{MRE}_{\mathsf{TriA}}$, $\mathsf{MRE}_{\mathsf{TriB}}$ (even smaller than 0.1 in most instances) compared to the baselines and validate our sampling-based secure triangle counting scheme. In contrast, AsgLDP consistently exhibits the highest $\mathsf{MRE}_{\mathsf{TriA}}$ and $\mathsf{MRE}_{\mathsf{TriB}}$ (even exceeding 0.5 in most instances). This is because AsgLDP does not optimize the structure of the synthetic graphs by incorporating triangle counts. While CAGM optimizes the structure of the synthetic graphs by incorporating triangle counts, it still cannot achieve lower $\mathsf{MRE}_{\mathsf{TriA}}$ and $\mathsf{MRE}_{\mathsf{TriB}}$ than PrivAGM. This is because CAGM does not consider the inherent directionality of the edges.

8.3 System Cost Evaluation

Note that we do not compare PrivAGM with the baselines in terms of system cost, as CAGM considers a centralized setting and AsgLDP has significant loss in the generated synthetic graphs. As the third phase of PrivAGM is generating synthetic graphs in the plaintext domain, which is rapid, its cost is not included in our evaluation.

Local neighbor list encryption. Fig. 7 presents the results of the lower bound ε_E for leakage in local neighbor list encryption and the savings on the resulting ciphertext size, for varying the selective probability $p \in \{0.1, 0.2, 0.3, 0.4, 0.5, 1\}$. The results show that PrivAGM achieves a significant reduction in the resulting ciphertext

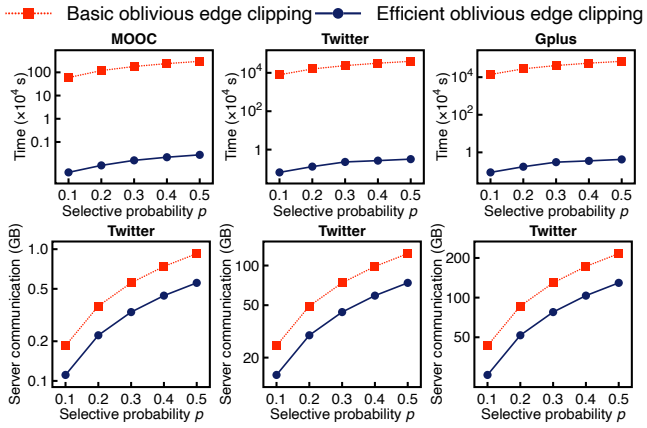


Figure 8: Comparison of basic and efficient oblivious edge clipping, for varying p.

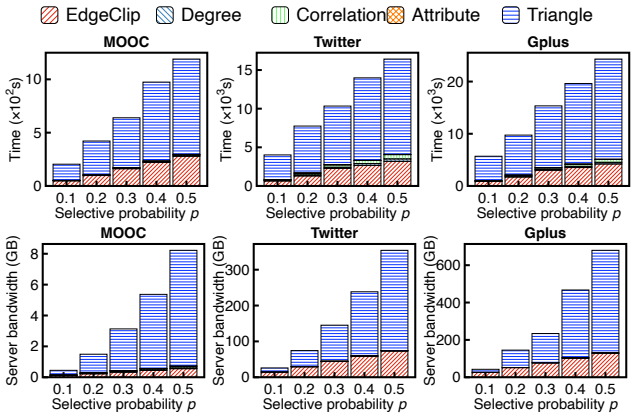


Figure 9: Time and communication cost of differentially private dAGM construction and their breakdown, for varying p.

size compared to direct encryption of the complete local neighbor list in dense encoding (up to 90% under $p = 0.1, \varepsilon_E \approx 2.3$). Moreover, the savings on the resulting ciphertext size remain consistent across different datasets, highlighting the scalability of our local neighbor list encryption protocol. These savings will significantly reduce the system cost in the subsequent secure dAGM construction.

Oblivious edge clipping. Fig. 8 presents a comparison of basic and efficient oblivious edge clipping in terms of time and communication cost. The results for the basic protocol are estimated from processing 1000 encrypted edges, as it is significantly slow (taking a few days). The comparison shows that the efficient protocol achieves a substantial speedup compared to the basic protocol (up to 10000×). Additionally, the efficient protocol achieves approximately 50% savings in communication cost compared to the basic protocol. These performance improvements stem from the fact that the efficient protocol allows the aggregators to process the degree of each vertex in parallel and obliviously clip the edges in batches.

Differentially private dAGM construction. Fig. 9 presents the time and communication cost of differentially private dAGM construction and their breakdown. We can observe that the majority of the cost can be attributed to the secure triangle counting. Its high time cost is primarily due to the communication required by the 3-input multiplication gates (i.e., lines 6 and 14 of Algorithm 5). Additionally, the results further demonstrate that our local neighbor list encryption can significantly save the cost of secure dAGM construction. While the cost of differentially private dAGM construction is relatively high, it is a *one-off* process. Once the dAGM is constructed, it can generate numerous synthetic graphs.

9 RELATED WORK

9.1 Private Centralized Graph Analytics

In the *centralized* setting, there exists a considerable body of work on developing methods for private graph analysis. In this setting, a curator possesses the complete graph and publishes some noisy information in a manner that satisfies DP. Some works publish differentially private statistics of the original graph, e.g., degree distribution [35] and subgraph counting [11]. Other works [5, 25, 45] develop differentially private generative graph models. Specifically, the work [45] focuses on *unattributed* graphs and only capture the structural information of the original graphs. In contrast, the works [5, 25] center on *attributed* graphs and consider capturing the attribute-edge correlation of the original graphs. Chen *et al.* [5] propose CAGM, an extension of [25], which additionally captures the community structure of the original graph.

However, since the above methods are designed for the centralized setting, they are not applicable in our decentralized setting. Moreover, the mentioned methods focus exclusively on *undirected* graphs, making them unsuitable for achieving satisfactory utility on directed graphs, as demonstrated in our experiments.

9.2 Private Decentralized Graph Analytics

Another line of works considers the more challenging *decentralized* setting, where the graph is presented in a decentralized manner, and no party has direct access to the complete graph. Several works [22, 23, 36, 44] focus on extracting differentially private subgraph counts. Other works [3, 34, 43, 46] focus on constructing differentially private generative graph models over decentralized graphs. However, these methods [3, 34, 46] fail to capture the attributed information of the graph. Wei *et al.* [43] propose AsgLDP, which considers decentralized *attributed* graphs by incorporating the attribute-edge correlation into the framework of [34]. AsgLDP is the work most related to ours. However, since AsgLDP does not consider the inherent directionality of the edges in the original graph, its generated synthetic graphs have poor utility when dealing with directed graphs, as demonstrated in our experiments.

10 CONCLUSION

This paper presents PrivAGM, the first solution for the secure construction of differentially private dAGM on decentralized social graphs. By bridging the gap among edge DP, edge LDP, MPC, and generative graph models, PrivAGM enables aggregators to effectively construct differentially private dAGM on decentralized social graphs without compromising the individual privacy of the users. The evaluation results on three real-world graph datasets demonstrate that PrivAGM outperforms the state-of-the-art baselines, offering significantly higher utility in synthetic graphs. We believe that this work lays a solid foundation for advancing research in the secure construction of generative graph models on decentralized social graphs and the synthesis of directed attributed graphs.

REFERENCES

[1] Jeremiah Blocki, Avrim Blum, Anupam Datta, and Or Sheffet. 2013. Differentially private data analysis of social networks via restricted sensitivity. In *Proc. of ACM ITCS*.

[2] Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Nishant Kumar, and Mayank Rathee. 2021. Function Secret Sharing for Mixed-Mode and Fixed-Point Secure Computation. In *Proc. of EUROCRYPT*.

[3] Felipe T Brito, Victor AE Farias, Cheryl Flynn, Subhabrata Majumdar, Javam C Machado, and Divesh Srivastava. 2023. Global and Local Differentially Private Release of Count-Weighted Graphs. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–25.

[4] Umit Can and Bilal Alatas. 2019. A new direction in social network analysis: Online social network analysis problems and applications. *Physica A: Statistical Mechanics and its Applications* 535 (2019), 122372.

[5] Xihui Chen, Sjouke Mauw, and Yunior Ramírez-Cruz. 2020. Publishing community-preserving attributed social graphs with a differential privacy guarantee. *Proceedings on Privacy Enhancing Technologies* 2020, 4 (2020), 131–152.

[6] Fan Chung and Linyuan Lu. 2002. The average distances in random graphs with given expected degrees. *Proceedings of the National Academy of Sciences* 99, 25 (2002), 15879–15882.

[7] Michael Curtiss, Iain Becker, Tudor Bosman, Sergey Doroshenko, Lucian Grijincu, Tom Jackson, Sandhya Kunnatur, Søren B. Lassen, Philip Pronin, Sriram Sankar, Guanghao Shen, Gintaras Woss, Chao Yang, and Ning Zhang. 2013. Unicorn: A System for Searching the Social Graph. *Proceedings of the VLDB Endowment* 6, 11 (2013), 1150–1161.

[8] Emma Dauterman, Mayank Rathee, Raluca Ada Popa, and Ion Stoica. 2022. Waldo: A Private Time-Series Database from Function Secret Sharing. In *Proc. of IEEE S&P*.

[9] Daniel Demmler, Thomas Schneider, and Michael Zohner. 2015. ABY - A Framework for Efficient Mixed-Protocol Secure Two-Party Computation. In *Proc. of NDSS*.

[10] Diaspora. 2023. The online social world where you are in control. online at https://diasporafoundation.org. [Online; Accessed 1-Jul-2023].

[11] Xiaofeng Ding, Shujun Sheng, Huajian Zhou, Xiaodong Zhang, Zhifeng Bao, Pan Zhou, and Hai Jin. 2021. Differentially private triangle counting in large graphs. *IEEE Transactions on Knowledge and Data Engineering* 34, 11 (2021), 5278–5292.

[12] Irit Dinur and Kobbi Nissim. 2003. Revealing information while preserving privacy. In *Proc. of ACM PODS*.

[13] Derek Doran. 2014. Triad-based role discovery for large social systems. In *Proc. of SocInfo*.

[14] Cynthia Dwork. 2006. Differential Privacy. In *Proc. of ICALP*.

[15] Reo Eriguchi, Atsunori Ichikawa, Noboru Kunihiro, and Koji Nuida. 2021. Efficient Noise Generation to Achieve Differential Privacy with Applications to Secure Multiparty Computation. In *Proc. of FC*.

[16] Saba Eskandarian and Dan Boneh. 2022. Clarion: Anonymous communication from multiparty shuffling protocols. In *Proc. of NDSS*.

[17] Fireblocks. 2023. Remove the complexity of working with digital assets. online at https://www.fireblocks.com. [Online; Accessed 1-Jun-2023].

[18] Michael Hay, Chao Li, Gerome Miklau, and David Jensen. 2009. Accurate estimation of the degree distribution of private networks. In *Proc. of IEEE ICDE*.

[19] Michael G Hay. 2010. *Enabling accurate analysis of private network data.* Ph. D. Dissertation. University of Massachusetts Amherst.

[20] Xi He, Ashwin Machanavajjhala, Cheryl J. Flynn, and Divesh Srivastava. 2017. Composing Differential Privacy and Secure Computation: A Case Study on Scaling Private Record Linkage. In *Proc. of ACM CCS*.

[21] Thomas Humphries, Rasoul Akhavan Mahdavi, Shannon Veitch, and Florian Kerschbaum. 2022. Selective MPC: Distributed Computation of Differentially Private Key-Value Statistics. In *Proc. of ACM CCS*.

[22] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2021. Locally Differentially Private Analysis of Graph Statistics. In *Proc. of USENIX Security*.

[23] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2022. Communication-efficient triangle counting under local differential privacy. In *Proc. of USENIX Security*.

[24] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2022. Differentially Private Triangle and 4-Cycle Counting in the Shuffle Model. In *Proc. of ACM CCS*.

[25] Zach Jorgensen, Ting Yu, and Graham Cormode. 2016. Publishing attributed social graphs with formal privacy guarantees. In *Proc. of ACM SIGMOD*.

[26] Shiva Prasad Kasiviswanathan, Homin K. Lee, Kobbi Nissim, Sofya Raskhodnikova, and Adam D. Smith. 2011. What Can We Learn Privately? *SIAM J. Comput.* 40, 3 (2011), 793–826.

[27] Srijan Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting dynamic embedding trajectory in temporal interaction networks. In *Proc. of ACM KDD*.

[28] Yehuda Lindell. 2017. How to Simulate It - A Tutorial on the Simulation Proof Technique. In *Tutorials on the Foundations of Cryptography*. 277–346.

[29] Julian J. McAuley and Jure Leskovec. 2012. Learning to Discover Social Circles in Ego Networks. In *Proc. of NeurIPS*.

[30] Meta. 2023. The value of secure multi-party computation. online at https://privacytech.fb.com/multi-party-computation/. [Online; Accessed 1-Jun-2023].

[31] MPCVault. 2023. Multisig crypto wallet for business. online at https://mpcvault.com. [Online; Accessed 1-Jun-2023].

[32] Arpita Patra, Thomas Schneider, Ajith Suresh, and Hossein Yalame. 2021. ABY2.0: Improved Mixed-Protocol Secure Two-Party Computation.. In *Proc. of USENIX Security*.

[33] Joseph J Pfeiffer III, Sebastian Moreno, Timothy La Fond, Jennifer Neville, and Brian Gallagher. 2014. Attributed graph models: Modeling network structure with correlated attributes. In *Proc. of ACM WWW*.

[34] Zhan Qin, Ting Yu, Yin Yang, Issa Khalil, Xiaokui Xiao, and Kui Ren. 2017. Generating Synthetic Decentralized Social Graphs with Local Differential Privacy. In *Proc. of ACM CCS*.

[35] Sofya Raskhodnikova and Adam Smith. 2016. Lipschitz extensions for node-private graph statistics and the generalized exponential mechanism. In *Proc. of IEEE FOCS*.

[36] Haipei Sun, Xiaokui Xiao, Issa Khalil, Yin Yang, Zhan Qin, Wendy Hui Wang, and Ting Yu. 2019. Analyzing Subgraph Statistics from Extended Local Views with Decentralized Differential Privacy. In *Proc. of ACM CCS*.

[37] Sijun Tan, Weikeng Chen, Ryan Deng, and Raluca Ada Popa. 2023. MPCAuth: Multi-factor Authentication for Distributed-trust Systems. In *Proc. of IEEE S&P*.

[38] Sijun Tan, Brian Knott, Yuan Tian, and David J Wu. 2021. CryptGPU: Fast Privacy-Preserving Machine Learning on the GPU. In *Proc. of IEEE S&P*.

[39] Songlei Wang, Yifeng Zheng, and Xiaohua Jia. 2023. The full version of PrivAGM. https://github.com/songleiW/PrivAGM/blob/main/PrivAGM_full_version.pdf.

[40] Zuan Wang, Xiaofeng Ding, Hai Jin, and Pan Zhou. 2022. Efficient secure and verifiable location-based skyline queries over encrypted data. *Proceedings of the VLDB Endowment* 15, 9 (2022), 1822–1834.

[41] Stanley L Warner. 1965. Randomized response: A survey technique for eliminating evasive answer bias. *J. Amer. Statist. Assoc.* 60, 309 (1965), 63–69.

[42] Duncan J Watts and Steven H Strogatz. 1998. Collective dynamics of 'small-world'networks. *Nature* 393, 6684 (1998), 440–442.

[43] Chengkun Wei, Shouling Ji, Changchang Liu, Wenzhi Chen, and Ting Wang. 2020. AsgLDP: collecting and generating decentralized attributed graphs with local differential privacy. *IEEE Transactions on Information Forensics and Security* 15 (2020), 3239–3254.

[44] Qingqing Ye, Haibo Hu, Man Ho Au, Xiaofeng Meng, and Xiaokui Xiao. 2022. LF-GDPR: A framework for estimating graph metrics with local differential privacy. *IEEE Transactions on Knowledge and Data Engineering* 34, 10 (2022), 4905–4920.

[45] Quan Yuan, Zhikun Zhang, Linkang Du, Min Chen, Peng Cheng, and Mingyang Sun. 2023. PrivGraph: Differentially Private Graph Data Publication by Exploiting Community Information. In *Proc. of USENIX Security*.

[46] Yuxuan Zhang, Jianghong Wei, Xiaojian Zhang, Xuexian Hu, and Wenfen Liu. 2018. A two-phase algorithm for generating synthetic graph under local differential privacy. In *Proc. of ACM ICCNS*.

A SECURE ATTRIBUTE DISTRIBUTION EXTRACTION

We present how $P_{1,2,3}$ securely extract the noisy version of attribute distribution Θ_F from $\{[\![f_i]\!]\}$, while ensuring that the extracted noisy distribution (denoted as $\widetilde{\Theta}_F$) is differentially private in the individual view of $P_{1,2,3}$. The design intuition is the same as that of Algorithm 4. Specifically, $P_{1,2,3}$ first securely extract the differentially private histogram of attributes, then derive $\widetilde{\Theta}_F$ from the histogram. Algorithm 8 gives our secure attribute extraction protocol. Here, the sensitivity Δ is set to 2, as modifying the attribute of a single vertex leads to changes in the frequency of two attributes within the social graph.

B PROOF OF THEOREM 7.1

Proof. Note that the secret shares $\{\langle b_{ij}\rangle\}$ contained within $[E_i]$ reveal nothing about the corresponding values. Hence, $[E_i]$ can be described as an observed N-dimensional bit vector $\mathbf{b} = (b_1, \cdots, b_N)$ for the adversary, where $b_j = 1, j \in [1, N]$ if $(i, j, \langle b_{ij}\rangle) \in [E_i]$, and otherwise $b_j = 0$.

We abuse $\mathcal{M}$ to denote the secure mechanisms in PrivAGM, and use $Pr[\mathbf{A}[i, j] \rightarrow b_j]$ to represent the probability that, given a bit $\mathbf{A}[i, j]$ of user i's local neighbor list, the adversary observes

Algorithm 8 Secure Attribute Distribution Extraction

Input: Encrypted attributes $\{[\![f_i]\!]\}$; privacy budget, parameter ε_F, δ_F.
Output: Differentially private attribute distribution $\widetilde{\Theta}_F$.

// $P_{1,2}$ operate on $\{[\![f_i]\!]\}$:
$[\![\mathbf{T}]\!] = [\,]$. // Empty table.
for all $[\![f_i]\!] \in \{[\![f_i]\!]\}$ **do**
  Append row $[\![f_i]\!] \| [\![\rho]\!]$ to $[\![\mathbf{T}]\!]$, where $[\![\rho]\!] = [\![1]\!]$.
end for
// P_3 constructs a dummy attribute value table $\mathbf{T}'$:
$\mathbf{T}' = [\,]$. // Empty table.
for all $f \in \mathbb{Z}_{2^L}$ **do**
  $n = \max(0, Lap(\varepsilon_F, \delta_F, 2))$. // Draw a noise from $Lap(\varepsilon_F, \delta_F, 2)$.
  Append n rows of $f \| \rho$ to $\mathbf{T}'$, where $\rho = 0$.
end for
P_3 secret-shares $\mathbf{T}'$ with $P_{1,2}$ to produce $[\![\mathbf{T}']\!]$.
// P_1, P_2 perform the remaining operations:
$[\![\hat{\mathbf{T}}]\!] = \frac{[\![\mathbf{T}]\!]}{[\![\mathbf{T}']\!]}$. // Vertical concatenation.
$[\![\hat{\mathbf{T}}']\!] = \text{secShuffle}([\![\hat{\mathbf{T}}]\!])$. //Oblivious row-wise shuffle.
Safely reveal column 1 of $[\![\hat{\mathbf{T}}']\!]$ to obtain $[\hat{\mathbf{T}}']$.
Initialize encrypted histogram $[\![\mathbf{H}]\!] = [\![0]\!]^{2^L}$.
for all row $f \| [\![\rho]\!]$ in $[\hat{\mathbf{T}}']$ **do**
  $[\![\mathbf{H}[f]]\!] = [\![\mathbf{H}[f]]\!] + [\![\rho]\!]$. // Index attribute value to histogram $[\![\mathbf{H}]\!]$.
end for
$[\![\widetilde{\mathbf{H}}[f]]\!] += [\![Lap(\varepsilon_F, 0, 2)]\!], f \in \mathbb{Z}_{2^L}$.
Safely reveal the differentially private histogram $\widetilde{\mathbf{H}}$.
$\widetilde{\mathbf{H}}'[f] = \max(0, \widetilde{\mathbf{H}}[f]), f \in \mathbb{Z}_{2^L}$. // Truncation.
$\widetilde{\mathbf{H}}''[f] = \frac{\widetilde{\mathbf{H}}'[f]}{\text{sum}(\widetilde{\mathbf{H}}')}, f \in \mathbb{Z}_{2^L}$. // Normalization
Initialize the attribute distribution $\widetilde{\Theta}_F = \{0\}^{2^L}$.
$\widetilde{\Theta}_F(f) = \widetilde{\mathbf{H}}''[f], f \in \mathbb{Z}_{2^L}$.
return Differentially private attribute distribution $\widetilde{\Theta}_F$.

$b_j \in \{0, 1\}$. The values of $Pr[\mathbf{A}[i, j] \to b_j]$ can be categorized into four cases:

- $\mathbf{A}[i, j] = 0$ and $b_j = 0$: This case corresponds to the scenario where either user i does not include the zero bit $\mathbf{A}[i, j]$ in E_i (with a probability of $1-p$), or user i includes $\mathbf{A}[i, j]$ in E_i (with a probability of p), but does not choose the adversary to receive $(i, j, \langle 0 \rangle)$ (with a probability of $1/3$). Therefore, $Pr[0 \to 0] = 1-p+p \cdot 1/3 = 1-2p/3$.
- $\mathbf{A}[i, j] = 1$ and $b_j = 0$: This case corresponds to the scenario where user i includes the nonzero bit $\mathbf{A}[i, j]$ in E_i (with a probability of 1), but does not choose the adversary to receive $(i, j, \langle 1 \rangle)$ (with a probability of $1/3$). Therefore, $Pr[1 \to 0] = 1 \cdot 1/3 = 1/3$.
- $\mathbf{A}[i, j] = 1$ and $b_j = 1$: This case corresponds to the scenario where user i includes the nonzero bit $\mathbf{A}[i, j]$ in E_i (with a probability of 1) and also chooses the adversary to receive $(i, j, \langle 1 \rangle)$ (with a probability of $2/3$). Therefore, $Pr[1 \to 1] = 1 \cdot 2/3 = 2/3$.
- $\mathbf{A}[i, j] = 0$ and $b_j = 1$: This case corresponds to the scenario where user i includes the zero bit $\mathbf{A}[i, j]$ in E_i (with a probability of p) and also chooses the adversary to receive $(i, j, \langle 0 \rangle)$ (with a probability of $2/3$). Therefore, $Pr[0 \to 1] = p \cdot 2/3 = 2p/3$.

Given any two local neighbor lists $\mathbf{A}[i, :]$ and $\mathbf{A}'[i, :]$ that differ in one bit, we need to prove that $\frac{Pr[\mathcal{M}(\mathbf{A}[i,:])=\mathbf{b}]}{Pr[\mathcal{M}(\mathbf{A}'[i,:])=\mathbf{b}]} \le e^{\varepsilon_E}$. Without loss of generality, let us assume that $\mathbf{A}[i, 1]$ and $\mathbf{A}'[i, 1]$ are not identical. Then we have:

$$\frac{Pr[\mathcal{M}(\mathbf{A}[i,:]) = \mathbf{b}]}{Pr[\mathcal{M}(\mathbf{A}'[i,:]) = \mathbf{b}]} = \frac{Pr[\mathbf{A}[i,1] \to b_1] \cdots Pr[\mathbf{A}[i,N] \to b_N]}{Pr[\mathbf{A}'[i,1] \to b_1] \cdots Pr[\mathbf{A}'[i,N] \to b_N]} = \frac{Pr[\mathbf{A}[i,1] \to b_1]}{Pr[\mathbf{A}'[i,1] \to b_1]}. \tag{3}$$

We analyze Eq. 3 in four cases:
1) $\frac{Pr[0\to0]}{Pr[1\to0]} = \frac{1-2p/3}{1/3} = 3-2p$; 2) $\frac{Pr[1\to0]}{Pr[0\to0]} = \frac{1/3}{1-2p/3} = \frac{1}{3-2p}$;
3) $\frac{Pr[0\to1]}{Pr[1\to1]} = \frac{2p/3}{2/3} = p$; 4) $\frac{Pr[1\to1]}{Pr[0\to1]} = \frac{2/3}{2p/3} = 1/p$.
We then need to find the upper bound (i.e., e^{ε_E}) of the set $\{3-2p, \frac{1}{3-2p}, p, 1/p\}$. Since $0 \le p \le 1$, we have $1/p \ge p$ and $3-2p \ge \frac{1}{3-2p}$. Therefore, we can conclude that $\{3-2p, \frac{1}{3-2p}, p, 1/p\} \le \max\{1/p, 3-2p\} = e^{\varepsilon_E}$, and $\varepsilon_E = \max\{\ln(1/p), \ln(3-2p)\}$. The probability that the adversary views the same local neighbor list $\mathbf{b}$ from $\mathbf{A}[i,:], \mathbf{A}'[i,:]$ is bounded by ε_E, which satisfies ε_E-edge LDP based on Definition 2.1. □

C PROOF OF THEOREM 7.2

Proof. Note that only $P_{1,2}$ have access to $[\hat{\mathbf{T}}'_{\mathcal{D}}]$, and therefore we only analyze the view of the adversary who controls either P_1 or P_2. Since the secret shares (i.e., the 3nd column of table $[\hat{\mathbf{T}}'_{\mathcal{D}}]$) reveal nothing about the corresponding values, $[\hat{\mathbf{T}}'_{\mathcal{D}}]$ can be described as a histogram (denoted as $\mathbf{H}_{\mathcal{D}}$) of degree pairs observed by the adversary. In $\mathbf{H}_{\mathcal{D}}$, each bin $\mathbf{H}_{\mathcal{D}}[d^+, d^-], d^+, d^- \in [0, k]$ represents the (noisy) number of vertices that have degree pair (d^+, d^-). Then we proceed to prove that with probability $1-\delta_{\mathcal{D}}$, the probability for the adversary viewing the same $\mathbf{H}_{\mathcal{D}}$ from two neighboring graphs G and G' is bounded by $e^{\varepsilon_{\mathcal{D}}}$.

If all the noise (drawn in line 8 of Algorithm 3) for the degree pairs are non-negative, then the probability to output the same noisy histogram $\mathbf{H}_{\mathcal{D}}$ from G, G' is $\frac{Pr[\mathcal{M}(G)=\mathbf{H}_{\mathcal{D}}]}{Pr[\mathcal{M}(G')=\mathbf{H}_{\mathcal{D}}]}$:

$$\frac{\prod_{d^+,d^- \in [0,k]} Pr[\mathbf{H}_{\mathcal{D}}[d^+, d^-] - \mathbf{H}_G[d^+, d^-]]}{\prod_{d^+,d^- \in [0,k]} Pr[\mathbf{H}_{\mathcal{D}}[d^+, d^-] - \mathbf{H}_{G'}[d^+, d^-]]}, \tag{4}$$

where $\mathbf{H}_G[d^+, d^-]$ is the number of vertices in graph G that have degree pair (d^+, d^-). $Pr[\mathbf{H}_{\mathcal{D}}[d^+, d^-] - \mathbf{H}_G[d^+, d^-]]$ is the probability drawing the noise $\mathbf{H}_{\mathcal{D}}[d^+, d^-] - \mathbf{H}_G[d^+, d^-]$ from the Laplace distribution $Lap(\varepsilon_{\mathcal{D}}, \delta_{\mathcal{D}}, 4)$. Let D_Δ be the set of bins where $\mathbf{H}_G$ and $\mathbf{H}_G$ are not equal. Then Eq. 4:

$$\begin{aligned}
&\frac{\prod_{(d^+,d^-) \in D_\Delta} e^{\frac{-\varepsilon_{\mathcal{D}}}{4} \cdot |\mathbf{H}_{\mathcal{D}}[d^+,d^-] - \mathbf{H}_G[d^+,d^-] - \mu|}}{\prod_{(d^+,d^-) \in D_\Delta} e^{\frac{-\varepsilon_{\mathcal{D}}}{4} \cdot |\mathbf{H}_{\mathcal{D}}[d^+,d^-] - \mathbf{H}_{G'}[d^+,d^-] - \mu|}} \\
&= \prod_{(d^+,d^-) \in D_\Delta} e^{\frac{\varepsilon_{\mathcal{D}}}{4} \cdot \left(|\mathbf{H}_{\mathcal{D}}[d^+,d^-] - \mathbf{H}_{G'}[d^+,d^-] - \mu| - |\mathbf{H}_{\mathcal{D}}[d^+,d^-] - \mathbf{H}_G[d^+,d^-] - \mu|\right)} \\
&= e^{\frac{\varepsilon_{\mathcal{D}}}{4} \cdot \sum_{(d^+,d^-) \in D_\Delta} |\mathbf{H}_{\mathcal{D}}[d^+,d^-] - \mathbf{H}_{G'}[d^+,d^-] - \mu| - |\mathbf{H}_{\mathcal{D}}[d^+,d^-] - \mathbf{H}_G[d^+,d^-] - \mu|} \\
&\le e^{\frac{\varepsilon_{\mathcal{D}}}{4} \cdot \sum_{(d^+,d^-) \in D_\Delta} |\mathbf{H}_G[d^+,d^-] - \mathbf{H}_{G'}[d^+,d^-]|} \le e^{\frac{\varepsilon_{\mathcal{D}}}{4} \cdot 4} = e^{\varepsilon_{\mathcal{D}}}.
\end{aligned} \tag{5}$$

The above proof is based on a fact that as G and G' differ by the presence or absence of a single edge, the maximum discrepancy in the values of their degree pair histograms is 4, i.e., $\sum_{(d^+,d^-) \in D_\Delta} |\mathbf{H}_G[d^+, d^-] - \mathbf{H}_{G'}[d^+, d^-]| \le 4$.

The probability to draw a negative noise from $Lap(\varepsilon_{\mathcal{D}}, \delta_{\mathcal{D}}, 4)$ is [20]: $Pr[n < 0] = \sum_{x=-1}^{-\infty} \frac{e^{\frac{\varepsilon_{\mathcal{D}}}{4}}-1}{e^{\frac{\varepsilon_{\mathcal{D}}}{4}}+1} \cdot e^{\frac{-\varepsilon_{\mathcal{D}} \cdot |x-\mu|}{4}} = \frac{e^{\frac{-\mu \cdot \varepsilon_{\mathcal{D}}}{4}}}{e^{\frac{\varepsilon_{\mathcal{D}}}{4}}+1}$. Given $\mu = -\frac{4}{\varepsilon_{\mathcal{D}}} \cdot \ln((e^{\frac{\varepsilon_{\mathcal{D}}}{4}}+1) \cdot (1-(1-\delta_{\mathcal{D}})^{\frac{1}{4}}))$, we have $Pr[n < 0] = 1 - (1-\delta_{\mathcal{D}})^{\frac{1}{4}}$. As there are at most 4 bins in $\mathbf{H}_G, \mathbf{H}_{G'}$ that are not equal. Hence, the overall failing probability is $1-(1-Pr[n<0])^4 = \delta_{\mathcal{D}}$. This means that with probability $1-\delta_{\mathcal{D}}$, the probability for the adversary viewing the same $\mathbf{H}_{\mathcal{D}}$ from G, G' is bounded by $e^{\varepsilon_{\mathcal{D}}}$, which satisfies $(\varepsilon_{\mathcal{D}}, \delta_{\mathcal{D}})$-edge DP based on Definition 2.2.

Then we proceed to prove that the probability for the adversary viewing the same $\widetilde{\mathbf{H}}_{\mathcal{D}}$ from G, G' is bounded by $e^{\varepsilon_{\mathcal{D}}}$. The probability to output the same $\widetilde{\mathbf{H}}_{\mathcal{D}}$ from G and G' is bounded by $\frac{Pr[\mathcal{M}(G)=\widetilde{\mathbf{H}}_{\mathcal{D}}]}{Pr[\mathcal{M}(G')=\widetilde{\mathbf{H}}_{\mathcal{D}}]} \leq e^{\varepsilon_{\mathcal{D}}}$. The proof follows Eq. 5 directly. Since the noises added to $\widetilde{\mathbf{H}}_{\mathcal{D}}$ (line 19 of Algorithm 3) are drawn from $Lap(\varepsilon_{\mathcal{D}}, 0, 4)$ (i.e., $\delta = 0$), $\mathcal{M}$ satisfies pure $\varepsilon_{\mathcal{D}}$-edge DP based on Definition 2.2. □

D PROOF OF THEOREM 7.5

Proof. First, we establish the $\varepsilon_{\triangle}$-edge DP of $\widetilde{n}_{\triangle}^A$, and the same conclusion holds for $\widetilde{n}_{\triangle}^B$. Given two neighboring graphs G, G', we have $|n_{\triangle}^A - n_{\triangle}'^A| \leq k-1$, where $n_{\triangle}^A$ and $n_{\triangle}'^A$ represent the count of triangle TriA in G, G' respectively, after edge clipping with parameter k. The probability to output the same $\widetilde{n}_{\triangle}^A$ from G, G' is bounded by

$$\frac{Pr[\mathcal{M}(G) = \widetilde{n}_{\triangle}^A]}{Pr[\mathcal{M}(G') = \widetilde{n}_{\triangle}^A]} = \frac{Pr[n_{\triangle}^A - \widetilde{n}_{\triangle}^A]}{Pr[n_{\triangle}'^A - \widetilde{n}_{\triangle}^A]}$$
$$= \frac{e^{\frac{-\varepsilon_{\triangle} \cdot |n_{\triangle}^A - \widetilde{n}_{\triangle}^A|}{k-1}}}{e^{\frac{-\varepsilon_{\triangle} \cdot |n_{\triangle}'^A - \widetilde{n}_{\triangle}^A|}{k-1}}} = e^{\frac{\varepsilon_{\triangle}}{k-1} \cdot (|n_{\triangle}'^A - \widetilde{n}_{\triangle}^A| - |n_{\triangle}^A - \widetilde{n}_{\triangle}^A|)}$$
$$\leq e^{\frac{\varepsilon_{\triangle}}{k-1} \cdot (|n_{\triangle}'^A - n_{\triangle}^A|)} \leq e^{\varepsilon_{\triangle}}.$$

$\mathcal{M}$ satisfies $\varepsilon_{\triangle}$-edge DP for $\widetilde{n}_{\triangle}^A$ based on Definition 2.2. □

E PROOF OF THEOREM 7.7

Proof. Since the roles of $P_{\{1,2,3\}}$ in PrivAGM are symmetric, with the exception that $P_{1,2}$ receive the secret shares of dummy entities (i.e., $[\![\mathbf{T}']\!]$) from P_3, it is enough to analyze the existence of $\mathcal{S}$ under the condition that $\mathcal{A}$ corrupts P_1.

Simulator $\mathcal{S}$ in Colle**.** During this phase, P_1 exclusively receives the set of tuples $\{(i, j, \langle b_{ij} \rangle)\}$. Therefore, we can trivially construct the simulator $\mathcal{S}$ by invoking the simulator of ASS [9] and providing it with the output of $\mathcal{L}_E$ as input.

Simulator $\mathcal{S}$ in edgeClip**.** The efficient oblivious edge clipping protocol is presented in Algorithm 2. Initially, $\mathcal{A}$ possesses the clipped local neighbor lists $\{(i, j, \langle b_{ij} \rangle)\}$, k, and w. $\mathcal{A}$ only receives messages from other honest aggregators during the execution of lines 7, 10, **??**, and **??**. The operations in lines 7 and **??** are based on the FSS-based DCF from [2]. Hence, the simulator can be trivially constructed by invoking the simulator of FSS-based DCF [2] and providing it with $k/3$ as input. The operations in lines 10 and **??** rely on the basic secure multiplication operation in the ASS domain. Hence, the simulator can be trivially constructed by invoking the simulator of ASS [9].

Simulator $\mathcal{S}$ in extDeg**.** The secure degree sequence extraction protocol is given in Algorithm 3. Initially, $\mathcal{A}$ possesses the clipped local neighbor lists $\{(i, j, \langle b_{ij} \rangle)\}$, k, $\varepsilon_{\mathcal{D}}$, $\delta_{\mathcal{D}}$, and the secret shares of Laplacian noises. $\mathcal{A}$ only receives messages from other honest aggregators during the execution of lines 3, 11, 13, 14, and 20. The operations in lines 3 and 11 rely on the secret sharing operation in the ASS domain. Hence, the simulator can be trivially constructed by invoking the simulator of ASS [9]. The operation in line 13 is based on the oblivious shuffle protocol from [16]. Therefore, the simulator can be trivially constructed by invoking the simulator of oblivious shuffle [16]. During the execution of lines 14 and 20, $\mathcal{A}$ receives secret shares from P_2 to reveal $[\hat{\mathbf{T}}']$ and $\widetilde{\mathbf{H}}$. Based on the security of ASS, the secret shares from P_2 are uniformly random in the view of $\mathcal{A}$. For revealed $[\hat{\mathbf{T}}']$ and $\widetilde{\mathbf{H}}$, $\mathcal{S}$ adjusts P_2' shares such that the revealed values are indeed the coefficient vector it received from $\mathcal{F}$ (i.e., $\mathcal{L}_{\mathcal{D}}$).

Simulator $\mathcal{S}$ in extCorr **and** extAttr**.** As the analysis of the existence of $\mathcal{S}$ in extCorr and extAttr follows the same idea as that in extDeg, we omit the analysis here.

Simulator $\mathcal{S}$ in extTria**.** The secure triangle counts extraction protocol is given in Algorithm 5. Initially, $\mathcal{A}$ possesses $\{(i, j, \langle b_{ij} \rangle)\}$, k, $\varepsilon_{\mathcal{D}}$, $\delta_{\mathcal{D}}$, and the secret shares of Laplacian noises. $\mathcal{A}$ only receives messages from $P_{2,3}$ during the execution of lines 6, 14, and 22. The operations in lines 6 and 14 are based on the secure 3-input multiplication gate from [32]. Hence, the simulator can be trivially constructed by invoking the simulator of the secure 3-input multiplication gate [32]. During the execution of line 22, $\mathcal{A}$ receives secret shares from P_2 and P_3 to reveal $\widetilde{n}_{\triangle}^A$ and $\widetilde{n}_{\triangle}^B$. Based on the security of ASS, the secret shares from $P_{2,3}$ are uniformly random in the view of $\mathcal{A}$. For revealed $\widetilde{n}_{\triangle}^A, \widetilde{n}_{\triangle}^B$, $\mathcal{S}$ adjusts the shares of $P_{2,3}$ such that the revealed values are indeed the coefficient vector it received from $\mathcal{F}$ (i.e., $\mathcal{L}_{\triangle}$). □

F UTILITY METRICS

We evaluate the utility of the synthetic graphs using four widely-used metrics [5, 25, 34, 43].

• *Degree distribution:* We utilize the Kolmogorov-Smirnov statistic to evaluate how well a synthetic graph captures the degree distribution of the original graph (denoted as $\mathsf{KS}_{\mathsf{degree}}$), which quantifies the maximum distance between two distributions. Let $c_{\mathcal{D}^{\pm}}$ and $c'_{\mathcal{D}^{\pm}}$ represent the cumulative distribution functions estimated from the sorted degree pair sequences of the input and synthetic graphs, respectively. Then $\mathsf{KS}_{\mathsf{degree}} = \max_{d^+=0}^{d^+_{max}} \max_{d^-=0}^{d^-_{max}} |c_{\mathcal{D}^{\pm}}(d^+, d^-) - c'_{\mathcal{D}^{\pm}}(d^+, d^-)|$, where d^+_{max}/d^-_{max} are the maximum out/in-degree in the input and synthetic graphs. $c_{\mathcal{D}^{\pm}}(d^+, d^-)$ (and $c'_{\mathcal{D}^{\pm}}(d^+, d^-)$) represents the percentage of vertices in the input (and synthetic) graph whose out-degrees $\leq d^+$ and in-degrees $\leq d^-$. As $\mathsf{KS}_{\mathsf{degree}}$ is less sensitive to differences in the tails of the degree distributions, we additionally report the Hellinger distance (denoted as $\mathsf{HD}_{\mathsf{degree}}$) between the two degree distributions $\Theta_{\mathcal{D}^{\pm}}$ and $\Theta'_{\mathcal{D}^{\pm}}$: $\mathsf{HD}_{\mathsf{degree}} = \frac{1}{\sqrt{2}} \cdot \sqrt{\sum_{d^+=0}^{d^+_{max}} \sum_{d^-=0}^{d^-_{max}} (\sqrt{\Theta_{\mathcal{D}^{\pm}}(d^+, d^-)} - \sqrt{\Theta'_{\mathcal{D}^{\pm}}(d^+, d^-)})^2}$.

• *Attribute-edge correlation distribution:* To evaluate how well a synthetic graph captures the attribute-edge correlations of the original graph, we report the mean relative error (MRE) between their attribute-edge correlation distributions (denoted

as $\mathsf{MRE}_{\mathsf{corr}}$), and the Hellinger distance: $\mathsf{HD}_{\mathsf{corr}} = \frac{1}{\sqrt{2}} \cdot \sqrt{\sum_{f=0}^{2^L} \sum_{f'=0}^{2^L} (\sqrt{\Theta_{\vec{X}}(f \to f')} - \sqrt{\Theta'_{\vec{X}}(f \to f')})^2}$.

• *Local clustering coefficient:* The local clustering coefficient C_i for a vertex i is determined by the proportion of edges between the vertices within its neighborhood divided by the number of edges that could potentially exist between them. In directed graphs, $C_i = \frac{|\{(j,k):j,k \in Ne_i,(j,k) \in E\}|}{(d_i^+ + d_i^-) \cdot (d_i^+ + d_i^- - 1)}$, where Ne_i is the set of vertex i's out/in-neighbors, and E is the edge set [42]. We report the MRE between the average of the local clustering coefficient of the synthetic and original graphs (denoted as $\mathsf{MRE}_{\mathsf{cluster}}$).

• *Triangle counts:* We report the MRE in the number of triangles, TriA and TriB, between the synthetic and original graphs (denoted as $\mathsf{MRE}_{\mathsf{TriA}}$ and $\mathsf{MRE}_{\mathsf{TriB}}$ respectively).